PRINCIPLES OF DATA SCIENCE - ASSIGNMENT

Gokul G (241801068)

In [ ]:
```
# 01 - NUMPY BASICS
# Gokul G (241801068)
# Create a NumPy array:
#numpy.array()
import numpy as np
arr = np.array([1, 2, 3, 4, 5])
print(arr)
```

```
[1 2 3 4 5]
```

In [ ]:
```
#Using arange & printing type
my_arr = np.arange(10)
print(my_arr)
print(type(my_arr))
```

```
[0 1 2 3 4 5 6 7 8 9]
<class 'numpy.ndarray'>
```

In [ ]:
```
my_list = list(range(10))
print (my_list)
print(type(my_list))
```

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
<class 'list'>
```

In [ ]:
```
# importing numpy package
import numpy as np
# importing system module
import sys
# declaring a list of 1000 elements
S= range (1000)
# printing size of each element of the list
print("Size of each element of list in bytes: ", sys.getsizeof(S))
# printing size of the whole list
print("Size of the whole list in bytes: ", sys.getsizeof(S)*len (S))
# declaring a Numpy array of 1000 elements
D= np.arange(1000)
# printing size of each element of the Numpy array
print("Size of each element of the Numpy array in bytes: ",D.itemsize)
# printing size of the whole Numpy array
print("Size of the whole Numpy array in bytes: ",D.size*D.itemsize)
print(type(S))
```

```
Size of each element of list in bytes:  48
Size of the whole list in bytes:  48000
Size of each element of the Numpy array in bytes:  8
Size of the whole Numpy array in bytes:  8000
<class 'range'>
```

In [ ]:
```
# 0-D Array
arr = np.array(42)
print(arr)

#1-D Array
import numpy as np
arr = np.array([1, 2, 3, 4, 5])
print(arr)
```

```
42
[1 2 3 4 5]
```

In [ ]:
```
#2-D Array
arr = np.array([[1, 2, 3], [4, 5, 6]])
print(arr)
```

```
[[1 2 3]
 [4 5 6]]
```

In [ ]:
```
#3-D Array
arr = np.array([[[1, 2, 3], [4, 5, 6]], [[1, 2, 3], [4, 5, 6]]])
print(arr)
```

```
[[[1 2 3]
  [4 5 6]]

 [[1 2 3]
  [4 5 6]]]
```

In [ ]:
```
#Check how many dimensions the arrays have:
a = np.array(42)
b = np.array([1, 2, 3, 4, 5])
c = np.array([[1, 2, 3], [4, 5, 6]])
d = np.array([[[1, 2, 3], [4, 5, 6]], [[1, 2, 3], [4, 5, 6]]])
print(a.ndim)
print (b.ndim)
print(c.ndim)
print(d.ndim)
```

```
0
1
2
3
```

In [ ]:
```
#Create an array with 5 dimensions and verify that it has 5 dimensions:
arr = np.array([1, 2, 3, 4], ndmin=5)
print(arr)
print('number of dimensions :', arr.ndim)
```

```
[[[[[1 2 3 4]]]]]
number of dimensions : 5
```

In [ ]:
```
# Printing shape, data type...
data= np.array([[1.5, -0.1, 3], [0, -3, 6.5]])
print("Shape= ", data.shape)
print("Type", data.dtype)
print("Data = ", data)
print("\nAns1 = ", data * 10)
print("\nAns2= ", data+data)
```

```
Shape=  (2, 3)
Type float64
Data =  [[ 1.5 -0.1  3. ]
 [ 0.  -3.   6.5]]

Ans1 =  [[ 15.  -1.  30.]
 [  0. -30.  65.]]

Ans2=  [[ 3.  -0.2  6. ]
 [ 0.  -6.  13. ]]
```

In [ ]:

```
# Creating an array of zeros
#zeros_arr = np.zeros(5,2) #error
zeros_arr = np.zeros((5,2))
eros_arr = np.zeros(5)
print(zeros_arr)
# Creating an array of zeros
zeros_arr = np.zeros((2,3))
print(zeros_arr)
#Checking the data type of the array elements
print("Data type of the array: ", zeros_arr.dtype)
# Creating an array of zeros with defined data types
zeros_arr_int = np.zeros((3,3), dtype=np.int16)
print("\nArray of zeros with integer elements: \n", zeros_arr_int)
```

```
[[0. 0.]
 [0. 0.]
 [0. 0.]
 [0. 0.]
 [0. 0.]]
[[0. 0. 0.]
 [0. 0. 0.]]
Data type of the array:  float64

Array of zeros with integer elements:
 [[0 0 0]
 [0 0 0]
 [0 0 0]]
```

In [ ]:
```
# Creating an array of ones
ones_arr = np.ones(6)
print(ones_arr)
# Creating an array of ones
ones_arr = np.ones((3,2,4)) # 3- depth, 2- rows, 4 -colums
print("\n", ones_arr)
print("\n ndim= ", ones_arr.ndim)
#Checking the data type of the array elements
print("\nData type of the array: ",ones_arr.dtype)
#Creating an array of ones with defined data types.
ones_arr_int = np.ones((2,2), dtype=np.int16)
print("\nArray of ones with integer elements: \n", ones_arr_int)
```

```
[1. 1. 1. 1. 1. 1.]

 [[[1. 1. 1. 1.]
  [1. 1. 1. 1.]]

 [[1. 1. 1. 1.]
  [1. 1. 1. 1.]]

 [[1. 1. 1. 1.]
  [1. 1. 1. 1.]]]

 ndim=  3

Data type of the array:  float64

Array of ones with integer elements:
 [[1 1]
 [1 1]]
```

In [ ]:
```
# Creating an empty array
empty_arr = np.empty((3,2))
print(empty_arr)
#Checking the data type of the array elements
print("\nData type of the array: ", empty_arr.dtype)
```

```
[[1. 1.]
 [1. 1.]
 [1. 1.]]

Data type of the array:  float64
```

In [ ]:
```
# Creating an identity matrix of order 3
identity_matrix = np. eye (3)
print (identity_matrix)
# Creating an identity matrix of order 4
identity_matrix = np.eye(4)
print("\n", identity_matrix)
```

```
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]

 [[1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [0. 0. 1. 0.]
 [0. 0. 0. 1.]]
```

In [ ]:
```
# 02 - Data Types for ndarrays
# Gokul G (241801068)
#dtype
import numpy as np
arr1 = np.array([1, 2, 3], dtype=np.float64)
arr2 = np.array([1, 2, 3], dtype=np.int32)
print(arr1.dtype)
print(arr2.dtype)
```

```
float64
int32
```

In [ ]:
```
#convert or cast an array from one data type to another using ndarray's
astype me
#int to float
arr = np.array([1, 2, 3])
print("Before converting: ", arr, arr.dtype)
new_arr = arr.astype('f')
print("After converting: ", new_arr, new_arr.dtype, "\n")

#convert or cast an array from one data type to another using ndarray's
astype me
#float to int
arr = np.array([1.1, 2.2, 3.3])
print("Before converting: ", arr, arr.dtype)
new_arr = arr.astype('i')
print("After converting: ", new_arr, new_arr.dtype)
```

```
Before converting:  [1 2 3] int64
After converting:  [1. 2. 3.] float32

Before converting:  [1.1 2.2 3.3] float64
After converting:  [1 2 3] int32
```

In [ ]:
```
#array of strings representing numbers, to convert them to numeric form:
import numpy as np
arr = np.array(['1.1', '2.2', '-3.3'])
print("Before converting: ", arr, arr.dtype)
new_arr = arr.astype('f')
print("After converting: ", new_arr, new_arr.dtype)
```

```
Before converting:  ['1.1' '2.2' '-3.3'] <U4
After converting:  [ 1.1  2.2 -3.3] float32
```

In [ ]:
```
#a string that cannot be converted to float64, a ValueError will be raised.
arr = np.array(['1.1', '2.2', 'abc'])
print("Before converting: ", arr, arr.dtype)
try:
    new_arr = arr.astype(np.float64)
    print("After converting: ", new_arr, new_arr.dtype)
except ValueError as e:
    print("Conversion failed:", e)
```

```
Before converting:  ['1.1' '2.2' 'abc'] <U3
Conversion failed: could not convert string to float: np.str_('abc')
```

In [ ]:
```
#You can also use another array's dtype attribute:
arr1 = np.arange(10)
arr2 = np.random.randn(10)
print("array 1\n",arr1, arr1.dtype)
print("Array 2 \n",arr2, arr2.dtype)
new_arr = arr1.astype(arr2.dtype)
print("New Array \n", new_arr, new_arr.dtype)
```

```
array 1
 [0 1 2 3 4 5 6 7 8 9] int64
Array 2
 [-0.21240222 -0.92841337  1.20315759  1.32758367  0.01973444 -0.79109441
 -1.05314249 -0.62167675  1.04136568  0.31879299] float64
New Array
 [0. 1. 2. 3. 4. 5. 6. 7. 8. 9.] float64
```

In [ ]:
```
# Create an array of strings
str_arr = np.array(['hello', 'world', 'numpy'])
print(str_arr)
```

```
['hello' 'world' 'numpy']
```

In [ ]:
```python
# 03 - Arithmetic with NumPy Arrays
# Gokul G (241801068)
#Arithmetic with NumPy Arrays
import numpy as np
a = np.array([1., 2., 3., 4.])
b = np.array([5., 6., 7., 8.])
print(a + b)
print(a - b)
print(a * b)
print(a / b)
print(a ** b)
```

```
[ 6.  8. 10. 12.]
[-4. -4. -4. -4.]
[ 5. 12. 21. 32.]
[0.2        0.33333333 0.42857143 0.5       ]
[1.0000e+00 6.4000e+01 2.1870e+03 6.5536e+04]
```

In [ ]:
```python
# prompt: #Arithmetic with NumPy Arrays with different shape
import numpy as np
a = np.array([[1., 2.], [3., 4.]])
b = np.array([5., 6., 7., 8.])
#print(a + b) # operands could not be broadcast together with shapes (4,) (2,5)
```

In [ ]:
```python
# Arithmetic operations with scalars propagate the scalar argument to each element in the array:
a= np.array([1., 2., 3., 4.])
print(a + 1) # [2. 3. 4. 5.]
print(a - 1) # [0. 1. 2. 3.]
print(a * 2) # [2. 4. 6. 8.]
print(a / 2) # [0.5 1. 1.5 2. ]
print(a ** 2) # [ 1. 4. 9. 16.]
print(1/a)
```

```
[2. 3. 4. 5.]
[0. 1. 2. 3.]
[2. 4. 6. 8.]
[0.5 1.  1.5 2. ]
[ 1.  4.  9. 16.]
[1.         0.5        0.33333333 0.25      ]
```

In [ ]:

```python
#reshape numpy array
import numpy as np
arr = np.arange(12)
newarr = arr.reshape(4, 3)
print(newarr)
newarr = arr.reshape((4, 3),order='F')
print("\n",newarr)
```

```
[[ 0  1  2]
 [ 3  4  5]
 [ 6  7  8]
 [ 9 10 11]]

 [[ 0  4  8]
 [ 1  5  9]
 [ 2  6 10]
 [ 3  7 11]]
```

In [ ]:

```python
# A multidimensional array can also be reshaped:
import numpy as np
arr = np.array([[1, 2, 3, 4,9,10], [5, 6, 7, 8,11,12]])
print(arr)
newarr = arr.reshape(3, 2, 2)
print("\n",newarr)
newarr1 = newarr.reshape(3, -1)
print("\n reshape(3,-1) \n",newarr1)
#-1: NumPy will automatically calculate how many columns are needed based on
the original array's size
```

```
[[ 1  2  3  4  9 10]
 [ 5  6  7  8 11 12]]

 [[[ 1  2]
  [ 3  4]]

 [[ 9 10]
  [ 5  6]]

 [[ 7  8]
  [11 12]]]

 reshape(3,-1)
 [[ 1  2  3  4]
 [ 9 10  5  6]
 [ 7  8 11 12]]
```

In [ ]:

```python
#flattening or raveling:
import numpy as np
original_array = np.array([[10, 20], [30, 40]])
print("Original array:\n", original_array)

# Using flatten() - makes a copy
flattened_array = original_array.flatten()
flattened_array[0] = 999
print("Flattened array:\n", flattened_array)
print("Original array:\n", original_array)

# Using ravel() - makes a view (if possible)
raveled_array = original_array.ravel()
raveled_array[1] = 888

# Print results
print("Raveled array:\n", raveled_array)
print("Original array:\n", original_array)
```

```
Original array:
 [[10 20]
 [30 40]]
Flattened array:
 [999  20  30  40]
Original array:
 [[10 20]
 [30 40]]
Raveled array:
 [ 10 888  30  40]
Original array:
 [[ 10 888]
 [ 30  40]]
```

In [ ]:

```python
# np.ravel()
import numpy as np

# Create a multi-dimensional array
arr = np.array([[1, 2, 3], [4, 5, 6]])

# Flatten the array using np.ravel()
r_arr = np.ravel(arr)
print("Flattened array:", r_arr) # Output: [1 2 3 4 5 6]

# Modify an element in the flattened array
r_arr[0] = 10

# Observe the change in the original array
print("Original array after modification:", arr) # Output: [[10 2 3] [ 4 5
6]]

# Create a multi-dimensional array
arr = np.array([[1, 2, 3], [4, 5, 6]])
f_arr = arr.flatten()
print("Flattened array:", f_arr)
f_arr[0] = 10
print("original array after flattening:", arr)
```

```
Flattened array: [1 2 3 4 5 6]
Original array after modification: [[10  2  3]
 [ 4  5  6]]
Flattened array: [1 2 3 4 5 6]
original array after flattening: [[1 2 3]
 [4 5 6]]
```

In [ ]:

```python
#numpy power operator arr ** 0.5
import numpy as np
arr = np.array([1, 4, 9, 16])
result = arr ** 0.5
print(result)
```

```
[1. 2. 3. 4.]
```

In [ ]:

```python
#numpy comparison operators

import numpy as np

# Create two NumPy arrays
arr1 = np.array([[1, 2], [3, 4]])
arr2 = np.array([[2, 1], [3, 5]])

# Element-wise comparison
greater_than = arr1 > arr2
less_than = arr1 < arr2
equal_to = arr1 == arr2

print("Greater than:\n", greater_than)
print("\nLess than:\n", less_than)
print("\nEqual to:\n", equal_to)

# Array-wise comparison
arr3 = np.array([1, 2, 3])
arr4 = np.array([1, 2, 3])
arr5 = np.array([1, 2, 4])

result1 = np.array_equal(arr3, arr4) # True
result2 = np.array_equal(arr3, arr5) # False

print("\nArray-wise comparison 1:", result1)
print("Array-wise comparison 2:", result2)
```

```
Greater than:
 [[False  True]
 [False False]]

Less than:
 [[ True False]
 [False  True]]

Equal to:
 [[False False]
 [ True False]]

Array-wise comparison 1: True
Array-wise comparison 2: False
```

In [ ]:
```
arr = np.arange(15)
arr.reshape((5, -1))
other_arr = np.ones((3, 5))
other_arr.shape

arr.reshape(other_arr.shape)
```

Out[30]:
```
array([[ 0,  1,  2,  3,  4],
       [ 5,  6,  7,  8,  9],
       [10, 11, 12, 13, 14]])
```

In [ ]:
```
# 04 - BASIC INDEXING AND SLICING
# Gokul G (241801068)
# numpy 1-d array slice is similar to list slicing program

import numpy as np
x = np.arange(10)
print("x= ",x)
# [0 1 2 3 4 5 6 7 8 9]
print("\n x[:5]= ",x[:5]) # first five elements
# [0 1 2 3 4]
print("\n x[5: ]= ",x[5:]) # elements after index 5
# [5 6 7 8 9]
print("\n x[4:7]= ", x[4:7]) # middle sub-array
# [4 5 6]
print("\n x[ :: 2]= ",x[ :: 2]) # every other element
# [0 2 4 6 8]
print("\n x[1 :: 2]= ",x[1 :: 2]) # every other element, starting at index 1
# [1 3 5 7 9]
print("\n x[ ::- 1]= ",x[ ::- 1]) # all elements, reversed
# [9 8 7 6 5 4 3 2 1 0]
print("\n x[5: :- 2]= ",x[5 ::- 2]) # reversed every other from index 5
# [5 3 1]
```

```
x=  [0 1 2 3 4 5 6 7 8 9]

 x[:5]=  [0 1 2 3 4]

 x[5: ]=  [5 6 7 8 9]

 x[4:7]=  [4 5 6]

 x[ :: 2]=  [0 2 4 6 8]

 x[1 :: 2]=  [1 3 5 7 9]

 x[ ::- 1]=  [9 8 7 6 5 4 3 2 1 0]

 x[5: :- 2]=  [5 3 1]
```

In [ ]:
```
#broadcasted
x = np. arange(10)
print(x)
x[5:] = 2
print("\n After broadcasting1",x)

x[ :: 2] = 100
print("\n After broadcasting2",x)
```

```
[0 1 2 3 4 5 6 7 8 9]

 After broadcasting1 [0 1 2 3 4 2 2 2 2 2]

 After broadcasting2 [100   1 100   3 100   2 100   2 100   2]
```

In [ ]:
```
# Create an array
x = np. arange(10)
print("Original array:", x)

# Create a view (slice) of the array
view = x[2:7]
print("View of the array:", view)

# Modify the view
view [1:4] = 99
print("Modified view:", view)
```

```
Original array: [0 1 2 3 4 5 6 7 8 9]
View of the array: [2 3 4 5 6]
Modified view: [ 2 99 99 99  6]
```

In [ ]:
```
# Print the original array to see the changes
print("Original array after modifying the view:", x)

#slice[:]
view[:] = -9
print("Modified view:", view)

# Print the original array to see the changes
print("Original array after modifying the view:", x)
```

```
Original array after modifying the view: [ 0  1  2 99 99 99  6  7  8  9]
Modified view: [-9 -9 -9 -9 -9]
Original array after modifying the view: [ 0  1 -9 -9 -9 -9 -9  7  8  9]
```

In [ ]:
```
# If you want a copy of a slice of an ndarray instead of a view,
#you will need to explicitly copy the array

# To create a copy of a slice, use the copy() method:
x_copy = x[2:7]. copy ()
x_copy [: ] = 100
print("Modified copy:", x_copy)

# The original array remains unchanged
print("Original array:", x)
```

```
Modified copy: [100 100 100 100 100]
Original array: [ 0  1 -9 -9 -9 -9 -9  7  8  9]
```

In [ ]:
```
#Multidimensional Arrays 2d
x2 = np.array([[12, 5, 2, 4],
[ 7, 6, 8, 8],
[ 1, 6, 7, 7]])
# each index in the array is itself a one-dimensional array
print(x2[0])

# individual elements can be accessed recursively.
# But NumPy provides a more convenient syntax:
# you can index the array using a comma-separated list of indices
print(x2[2,0]) # prints the first element of the last row
# 1
print(x2[1] [2]) # prints the third element of the second row
# 8
print(x2[2,-1]) # prints the last element of the last row
# 7
# If you only provide one index to a multidimensional array,
# it will return the row with that index.
print(x2[0])
```

```
[12  5  2  4]
1
8
7
[12  5  2  4]
```

In [ ]:
```
# Multidimensional Arrays 3d
x3 =np.array([[[12, 5, 2, 4],
[ 7, 6, 8, 8]],

[[ 1, 6, 7, 7],
[ 3, 4, 5, 9]]])
# each index in the array is itself a two-dimensional array
print(x3[0])
```

```
[[12  5  2  4]
 [ 7  6  8  8]]
```

In [ ]:

```
# individual elements can be accessed recursively.
# But NumPy provides a more convenient syntax:
# you can index the array using a comma-separated list of indices
print(x3[1,0,2]) # prints the third element of the first row of the second
2d array
# 7
print(x3[0] [1] [2]) # prints the third element of the second row of the
first 2d array
# 8
print(x3[1,-1,-1]) # prints the last element of the last row of the second
2d array
# 9
# If you only provide one index to a multidimensional array,
# it will return the row with that index.
print(x3[0])
```

```
7
8
9
[[12  5  2  4]
 [ 7  6  8  8]]
```

In [ ]:

```
# assign scalar values to x2 and x3

x2_old=x2.copy()
# assign scalar values to x2
x2[0] = 99
print("\n x2 = \n", x2)

x3_old=x3[1].copy()
# assign scalar values to x3
x3[1] =99
print("\n x3= \n", x3)

x3[1]=x3_old
print("\n x3 = \n", x3)
```

```
 x2 =
[[99 99 99 99]
 [ 7  6  8  8]
 [ 1  6  7  7]]

 x3=
[[[12  5  2  4]
  [ 7  6  8  8]]

 [[99 99 99 99]
  [99 99 99 99]]]

 x3 =
[[[12  5  2  4]
  [ 7  6  8  8]]

 [[ 1  6  7  7]
  [ 3  4  5  9]]]
```

In [ ]:

```
# slicing 2d array

# create a 2d array
x2 = np.array([[12, 5, 2, 4], [7, 6, 8, 8], [1, 6, 7, 71]])
print("x2= ",x2)

# slicing the 2d array
print("\n x2[:2, :3] = \n",x2[:2, :3]) # two rows, three columns
print("\n x2[:3, :: 2]=\n ",x2[:3, :: 2]) # three rows, every other column
print("\n x2[: :- 1, ::- 1]= \n", x2[: :- 1, ::- 1]) # all rows and columns,
reversed
print("\n x2[ ::- 1]= \n",x2[ ::- 1]) # all rows alone reversed
```

```
x2=  [[12  5  2  4]
 [ 7  6  8  8]
 [ 1  6  7 71]]

 x2[:2, :3] =
 [[12  5  2]
 [ 7  6  8]]

 x2[:3, :: 2]=
  [[12  2]
 [ 7  8]
 [ 1  7]]

 x2[: :- 1, ::- 1]=
 [[71  7  6  1]
 [ 8  8  6  7]
 [ 4  2  5 12]]

 x2[ ::- 1]=
 [[ 1  6  7 71]
 [ 7  6  8  8]
 [12  5  2  4]]
```

In [ ]:
```
import numpy as np
x=np.arange(24).reshape(2,3,4)
print(x)
print(x[:,0])
print(x[:,:,0])
print(x[ ... ,0])
```

```
[[[ 0  1  2  3]
  [ 4  5  6  7]
  [ 8  9 10 11]]

 [[12 13 14 15]
  [16 17 18 19]
  [20 21 22 23]]]
[[ 0  1  2  3]
 [12 13 14 15]]
[[ 0  4  8]
 [12 16 20]]
[[ 0  4  8]
 [12 16 20]]
```

In [ ]:
```
# You can pass multiple slices just like you can pass multiple indexes:
print("x2=\n",x2)
print("x2[:2, :: 2]= \n",x2[:2, :: 2])
print("x2[1,:2]=\n",x2[1, :2])
```

```
x2=
 [[12  5  2  4]
 [ 7  6  8  8]
 [ 1  6  7 71]]
x2[:2, :: 2]=
 [[12  2]
 [ 7  8]]
x2[1,:2]=
 [7 6]
```

In [ ]:

```
# 05 - BOOLEAN INDEXING
# Gokul G (241801068)
#Boolean Indexing
import numpy as np
a = np.arange(12).reshape(3,4)
print("a=\n", a)
b = a > 4
print("b=\n",b)
print("a[b]=\n",a[b]) # 1d array
a[b] = 1
print("a=\n",a)
```

```
a=
 [[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
b=
 [[False False False False]
 [False  True  True  True]
 [ True  True  True  True]]
a[b]=
 [ 5  6  7  8  9 10 11]
a=
 [[0 1 2 3]
 [4 1 1 1]
 [1 1 1 1]]
```

In [ ]:

```
# prompt: boolean indexing with string and float with explanation

import numpy as np

# Boolean indexing with strings
names = np.array(['Bob', 'Joe', 'Will', 'Bob', 'Will', 'Joe', 'Joe'])
data = np.arange(28).reshape(7, 4)
print(names)
print("data=\n", data)
print("names bool= \n", names == 'Bob') # returns a boolean array
print("names == 'Bob' \n", data [names == 'Bob']) # selects rows where names
== 'Bo
print("names == 'Bob',2: \n", data [names == 'Bob', 2:]) # selects rows
where names
print("names == 'Bob', 3 \n", data [names == 'Bob', 3]) # selects rows where
names =
```

```
['Bob' 'Joe' 'Will' 'Bob' 'Will' 'Joe' 'Joe']
data=
 [[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]
 [12 13 14 15]
 [16 17 18 19]
 [20 21 22 23]
 [24 25 26 27]]
names bool=
 [ True False False  True False False False]
names == 'Bob'
 [[ 0  1  2  3]
 [12 13 14 15]]
names == 'Bob',2:
 [[ 2  3]
 [14 15]]
names == 'Bob', 3
 [ 3 15]
```

In [ ]:
```python
# select all except bob
import numpy as np
names = np.array(['Bob', 'Joe', 'Will', 'Bob', 'Will', 'Joe', 'Joe'])
data = np.arange(28).reshape(7, 4)
print(data[names != 'Bob'])
print("\n", data[~(names == 'Bob')])
data[names != 'Bob'] =- 1
print("\ndata= \n",data)
```

```
[[ 4  5  6  7]
 [ 8  9 10 11]
 [16 17 18 19]
 [20 21 22 23]
 [24 25 26 27]]

 [[ 4  5  6  7]
 [ 8  9 10 11]
 [16 17 18 19]
 [20 21 22 23]
 [24 25 26 27]]

data=
 [[ 0  1  2  3]
 [-1 -1 -1 -1]
 [-1 -1 -1 -1]
 [12 13 14 15]
 [-1 -1 -1 -1]
 [-1 -1 -1 -1]
 [-1 -1 -1 -1]]
```

In [ ]:
```python
# boolean indexing with string and float with explanation

import numpy as np

# Boolean indexing with strings and floats
names = np.array(['Alice', 'Bob', 'Charlie', 'David'])
ages = np.array([25.5, 30, 28.2, 35])

# Select names of people older than 29
older_than_29 = ages > 29 #bool array
print("People older than 29:", names [older_than_29])

# Select ages of people named 'Alice' or 'Charlie'
names_to_select = (names == 'Alice') | (names == 'Charlie') #bool array
print("Ages of Alice and Charlie:", ages [names_to_select])
```

```
People older than 29: ['Bob' 'David']
Ages of Alice and Charlie: [25.5 28.2]
```

In [ ]:
```python
# city names, temperature arrays. select hot cities

import numpy as np
city_names = np.array(['New York', 'Los Angeles', 'Chicago', 'Houston'])
temperatures = np.array([25, 32, 28, 35])

# Define a threshold for "hot" cities
hot_threshold = 30

# Select cities with temperatures above the threshold
hot_cities_bool = temperatures > hot_threshold
hot_cities = city_names [hot_cities_bool]
print("Hot cities:", hot_cities)
```

```
Hot cities: ['Los Angeles' 'Houston']
```

In [ ]:
```python
#negative temperature cities mark 0

city_names = np.array(['New York', 'Los Angeles', 'Chicago', 'Houston'])
temperatures = np.array([-25, 32, -28, 35])

# Boolean indexing to find cities with negative temperatures
negative_temp_indices = temperatures < 0 #bool

# Set temperatures of negative temperature cities to 0
temperatures [negative_temp_indices] = 0
print("Updated temperatures:", temperatures)
```

```
Updated temperatures: [ 0 32  0 35]
```

In [ ]:

```
# Fancy Indexing
# Creating an array for demonstration
arr = np.arange(32).reshape(8, 4)
print("arr=\n",arr)

# Selecting specific rows and columns using integer arrays
print("\narr[[1, 5, 7, 2]]= \n", arr[[1, 5, 7, 2]])

# Selecting specific rows and columns using integer arrays
print("\narr[[1, 5, 7, 2], [0, 3, 1, 2]]= \n",arr[[1, 5, 7, 2], [0, 3, 1,
2]])

# Selecting elements at specific indices
print("\narr[[1, 5, 7, 2]][:, [0, 3, 1, 2]]= \n",arr[[1, 5, 7, 2]][:, [0, 3,
1, 2]])
print("\n arr[[-3, -5, -7]]\n",arr[[-3, -5, -7]])

# Modifying elements using fancy indexing
arr[[1, 5, 7, 2]] = 1
print("\narr= \n", arr)
```

```
arr=
 [[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]
 [12 13 14 15]
 [16 17 18 19]
 [20 21 22 23]
 [24 25 26 27]
 [28 29 30 31]]

arr[[1, 5, 7, 2]]=
 [[ 4  5  6  7]
 [20 21 22 23]
 [28 29 30 31]
 [ 8  9 10 11]]

arr[[1, 5, 7, 2], [0, 3, 1, 2]]=
 [ 4 23 29 10]

arr[[1, 5, 7, 2]][:, [0, 3, 1, 2]]=
 [[ 4  7  5  6]
 [20 23 21 22]
 [28 31 29 30]
 [ 8 11  9 10]]

 arr[[-3, -5, -7]]
 [[20 21 22 23]
 [12 13 14 15]
 [ 4  5  6  7]]

arr=
 [[ 0  1  2  3]
```

```
 [ 1  1  1  1]
 [ 1  1  1  1]
 [12 13 14 15]
 [16 17 18 19]
 [ 1  1  1  1]
 [24 25 26 27]
 [ 1  1  1  1]]
```

In [ ]:

```
# 06 - Transposing Arrays and Swapping Axes
# Gokul G (241801068)
# Arrays have the transpose method and also the special T attribute
import numpy as np
arr = np.arange(12).reshape((3, 4))
print(arr)
print("Transpose using .T")
print(arr.T)
print("Transpose using the transpose() function")
print(np.transpose(arr))
```

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
Transpose using .T
[[ 0  4  8]
 [ 1  5  9]
 [ 2  6 10]
 [ 3  7 11]]
Transpose using the transpose() function
[[ 0  4  8]
 [ 1  5  9]
 [ 2  6 10]
 [ 3  7 11]]
```

In [ ]:

```
arr1=np.transpose(arr,(1,0))
print(arr1)

arr2=np.transpose(arr,(0,1))
print(arr2)
```

```
[[ 0  4  8]
 [ 1  5  9]
 [ 2  6 10]
 [ 3  7 11]]
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
```

In [ ]:

```
#3d array
arr = np.arange(12).reshape((2, 3,2))
print(arr)
print("Transpose using .T")
arr3dT=arr.T
print(arr3dT) # column has become row
print(arr3dT.shape)
print("Transpose using the transpose() function")
print(np.transpose(arr))
```

```
[[[ 0  1]
  [ 2  3]
  [ 4  5]]

 [[ 6  7]
  [ 8  9]
  [10 11]]]
Transpose using .T
[[[ 0  6]
  [ 2  8]
  [ 4 10]]

 [[ 1  7]
  [ 3  9]
  [ 5 11]]]
(2, 3, 2)
Transpose using the transpose() function
[[[ 0  6]
  [ 2  8]
  [ 4 10]]

 [[ 1  7]
  [ 3  9]
  [ 5 11]]]
```

In [ ]:
```
arr3d1=np.transpose(arr,(1,0,2))
print(arr3d1)

arr3d2=np.transpose(arr, (0,1,2))
print(arr3d2)
```

```
[[[ 0  1]
  [ 6  7]]

 [[ 2  3]
  [ 8  9]]

 [[ 4  5]
  [10 11]]]
[[[ 0  1]
  [ 2  3]
  [ 4  5]]

 [[ 6  7]
  [ 8  9]
  [10 11]]]
```

In [ ]:
```
#dot product
import numpy as np
arr = np.arange(12).reshape((3, 4))
print("Dot product of arr and arr2:")
print(np.dot(arr, arr.T))
```

```
Dot product of arr and arr2:
[[ 14  38  62]
 [ 38 126 214]
 [ 62 214 366]]
```

In [ ]:
```
#swapaxes similarly returns a view on the data without making a copy.
import numpy as np
arr = np.arange(12).reshape((3, 4))
print(arr)
print("Swap axes 0 and 1:")
print(np.swapaxes(arr, 0, 1))
```

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
Swap axes 0 and 1:
[[ 0  4  8]
 [ 1  5  9]
 [ 2  6 10]
 [ 3  7 11]]
```

In [ ]:
```python
#matrix multiplication using np.multiply

import numpy as np
arr1 = np.array([[1, 2],
[3,4]])
arr2 = np.array([[5, 6],
[7,8]])

# Matrix multiplication using np.multiply (element-wise multiplication)
result = np.multiply(arr1, arr2)
print("Result of element-wise multiplication:")
print(result)
```

```
Result of element-wise multiplication:
[[ 5 12]
 [21 32]]
```

In [ ]:
```python
#matrix multiplication using np.matmult
# Matrix multiplication using np.matmul (matrix product)
result = np.matmul(arr1, arr2)
print("Result of matrix multiplication:")
print(result)
```

```
Result of matrix multiplication:
[[19 22]
 [43 50]]
```

In [ ]:
```python
#matrix multiplication using np.dot
# Matrix multiplication using np.dot (matrix product)
result = np.dot(arr1, arr2)
print("Result of matrix multiplication:")
print(result)
```

```
Result of matrix multiplication:
[[19 22]
 [43 50]]
```

In [ ]:
```
# 7 - INTRODUCTION TO PANDAS DATAFRAME
# Gokul G (241801068)
#importing
import pandas
mydataset = {
  'cars': ["BMW", "Volvo", "Ford"],
  'passings': [3, 7, 2]
}
myvar = pandas.DataFrame(mydataset)
print(myvar)
```

```
    cars  passings
0    BMW         3
1  Volvo         7
2   Ford         2
```

In [ ]:
```
# Creating a Series from a list
import pandas as pd
data = [10, 20, 30, 40, 50]
series = pd.Series (data)
print(series)
```

```
0    10
1    20
2    30
3    40
4    50
dtype: int64
```

In [ ]:
```
# array representation and index object of the Series via its values and
#index attributes, respectively:
import pandas as pd

# Creating a Series from a list
data = [10, 20, 30, 40, 50]
series = pd.Series (data)
print("Series Representation:\n", series)

# Get the array representation and index object
array_representation = series. values
index_object = series. index #index object
print("Array Representation:", array_representation)
print("Index Object:", index_object)
```

```
Series Representation:
 0    10
1    20
2    30
3    40
4    50
dtype: int64
Array Representation: [10 20 30 40 50]
Index Object: RangeIndex(start=0, stop=5, step=1)
```

In [ ]:
```
#Create a Series with an index identifying each data point
# with a label:
a = [1, 7, 2]
myvar = pd.Series(a, index = ["x", "y", "z"])
print(myvar)
```

```
x    1
y    7
z    2
dtype: int64
```

In [ ]:
```
#Create a Series with an index identifying each data point
# with a label:

data = [10, 20, 30, 40, 50]
index_labels = ['a', 'b', 'c', 'd', 'e']
series = pd.Series (data, index=index_labels)
print(series)
```

```
a    10
b    20
c    30
d    40
e    50
dtype: int64
```

In [ ]:
```
# Accessing elements using labels
print(series['b']) # Access element with label 'b'
print(series[ ['a', 'c', 'e']]) # Access elements with labels 'a', 'c', and
'e'
```

```
20
a    10
c    30
e    50
dtype: int64
```

In [ ]:
```
# Using NumPy functions or NumPy-like operations, such as filtering with a
boolea
# array, scalar multiplication, or applying math functions, will preserve
the ind
# link
# Filtering with a boolean array
gt=series > 25
print("series>25=\n",gt)
series_filtered = series [series > 25]
print("Filtered Series:\n", series_filtered)

# Scalar multiplication
series_multiplied = series * 2
print("\nSeries after Scalar Multiplication:\n", series_multiplied)
```

```
series>25=
 a    False
b    False
c     True
d     True
e     True
dtype: bool
Filtered Series:
 c    30
d    40
e    50
dtype: int64

Series after Scalar Multiplication:
 a     20
b     40
c     60
d     80
e    100
dtype: int64
```

In [ ]:
```python
# Applying a math function (e.g., square root)
import numpy as np
print("series= \n", series)
series_sqrt = np.sqrt(series)
print("\nSeries after Applying Square Root:\n", series_sqrt)
```

```
series=
 a    10
b    20
c    30
d    40
e    50
dtype: int64

Series after Applying Square Root:
 a    3.162278
b    4.472136
c    5.477226
d    6.324555
e    7.071068
dtype: float64
```

In [ ]:
```python
#Another way to think about a Series is as a fixed-length, ordered dict, as it is
# of index values to data values.
# Creating a Series from a dictionary
population_dict = {'California': 38332521,
'Texas': 26448193,
'New York': 19651127,
'Florida': 19552860,
'Illinois': 12882135}
population = pd.Series(population_dict)
print(population)
```

```
California    38332521
Texas         26448193
New York      19651127
Florida       19552860
Illinois      12882135
dtype: int64
```

In [ ]:
```python
#using in operator
print('California' in population) #check if 'California' is present in population
print('Canada' in population) #check if 'Canada' is present in population series
```

```
True
False
```

In [ ]:

```python
#series with NAN(not a number) values
# Creating a Series with NaN values
data = [10, np.nan, 30, np.nan, 50]
index_labels = ['a', 'b', 'c', 'd', 'e']
series_with_nan = pd.Series(data, index=index_labels)
print(series_with_nan)

# isnull and notnull methods
print("\nChecking for NaN values:\n", series_with_nan.isnull())
print("\nChecking for non-NaN values:\n", series_with_nan.notnull())
```

```
a    10.0
b     NaN
c    30.0
d     NaN
e    50.0
dtype: float64

Checking for NaN values:
 a    False
b     True
c    False
d     True
e    False
dtype: bool

Checking for non-NaN values:
 a     True
b    False
c     True
d    False
e     True
dtype: bool
```

In [ ]:

```
#A useful Series feature for many applications is that it automatically
aligns by
# label in arithmetic operations:
# Arithmetic operations with automatic alignment
area_dict = {'California': 423967, 'Texas': 695662, 'New York': 141297,
'Florida': 170312, 'Illinois': 149995, 'North Carolina': 123456}
area = pd. Series(area_dict)
print("Population Series:\n", population)
print("\nArea Series:\n", area)
# Calculate population density
population_density = population / area
print("\nPopulation Density:\n", population_density)
```

```
Population Series:
 California    38332521
Texas         26448193
New York      19651127
Florida       19552860
Illinois      12882135
dtype: int64

Area Series:
 California        423967
Texas             695662
New York          141297
Florida           170312
Illinois          149995
North Carolina    123456
dtype: int64

Population Density:
 California         90.413926
Florida           114.806121
Illinois           85.883763
New York          139.076746
North Carolina           NaN
Texas              38.018740
dtype: float64
```

In [ ]:
```
# Find total population
# Male and female population (example data)
male_population_dict = {'California': 19000000, 'Texas': 13000000, 'New
York': 95,
'Florida': 9600000, 'Illinois': 6300000}
female_population_dict = {'California': 19332521, 'Texas': 13448193, 'New
York': 13456667,
'Florida': 9952860, 'Illinois': 6582135}

male_population = pd.Series(male_population_dict)
female_population = pd.Series(female_population_dict)

# Calculate total population
total_population = male_population + female_population
print("\nTotal Population:\n", total_population)
```

```
Total Population:
 California    38332521
Texas         26448193
New York      13456762
Florida       19552860
Illinois      12882135
dtype: int64
```

In [ ]:
```
# Both the Series object itself and its index have a name attribute, which
integr
# other key areas of pandas functionality:
# Naming the Series and its index
total_population.name = 'population'
total_population. index.name = 'state'
print(total_population)
```

```
state
California    38332521
Texas         26448193
New York      13456762
Florida       19552860
Illinois      12882135
Name: population, dtype: int64
```

In [ ]:

```
#A Series's index can be altered in-place by assignment:
# A Series's index can be altered in-place by assignment:
obj = pd.Series([4.5, 7.2, -5.3, 3.6], index=['d', 'b', 'a', 'c'])
print(obj)
obj2 = obj.reindex(['a', 'b', 'c', 'd', 'e'])
print(obj2)
```

```
d    4.5
b    7.2
a   -5.3
c    3.6
dtype: float64
a   -5.3
b    7.2
c    3.6
d    4.5
e    NaN
dtype: float64
```

In [ ]:

```python
# 5 subjects marks as separate series, find total marks for each student,
mention

import pandas as pd

# Sample marks for 5 subjects and 7 students
students = [f'Student {i}' for i in range(1, 8)]
subject1 = pd.Series([80, 75, 92, 68, 70, 95, 88], index=students)
subject2 = pd.Series([72, 88, 79, 91, 65, 82, 76], index=students)
subject3 = pd.Series([90, 68, 85, 73, 78, 92, 80], index=students)
subject4 = pd.Series([65, 92, 82, 88, 75, 79, 85], index=students)
subject5 = pd.Series([78, 70, 95, 62, 83, 86, 91], index=students)

# Calculate total marks for each student
total_marks = subject1 + subject2 + subject3 + subject4 + subject5
print("Total marks= \n", total_marks)
# Print total marks with subject numbers
print("\nTotal Marks for Each Student:")
for i, (student, marks) in enumerate(total_marks.items()):
  print(f"Student {i+1} ({student}) : {marks}")
```

```
Total marks=
 Student 1    385
Student 2    393
Student 3    433
Student 4    382
Student 5    371
Student 6    434
Student 7    420
dtype: int64

Total Marks for Each Student:
Student 1 (Student 1) : 385
Student 2 (Student 2) : 393
Student 3 (Student 3) : 433
Student 4 (Student 4) : 382
Student 5 (Student 5) : 371
Student 6 (Student 6) : 434
Student 7 (Student 7) : 420
```

In [ ]:
```python
# 8 - DATAFRAME
# Gokul G (241801068)
#Create a simple Pandas DataFrame:
import pandas as pd
data = {
  "calories": [420, 380, 390],
  "duration": [50, 40, 45]
}
#load data into a DataFrame object:
df = pd.DataFrame(data)
print(df)
```

```
   calories  duration
0       420        50
1       380        40
2       390        45
```

In [ ]:
```python
data = {'state': ['Ohio', 'Ohio', 'Ohio', 'Nevada', 'Nevada', 'Nevada'],
'year': [2000, 2001, 2002, 2001, 2002, 2003],
'pop': [1.5, 1.7, 3.6, 2.4, 2.9, 3.2]}
frame = pd.DataFrame(data)
print(frame)
```

```
    state  year  pop
0    Ohio  2000  1.5
1    Ohio  2001  1.7
2    Ohio  2002  3.6
3  Nevada  2001  2.4
4  Nevada  2002  2.9
5  Nevada  2003  3.2
```

In [ ]:
```python
#Change the order of colums
frame = pd.DataFrame(data, columns=['year', 'state', 'pop'])
print(frame)
```

```
   year   state  pop
0  2000    Ohio  1.5
1  2001    Ohio  1.7
2  2002    Ohio  3.6
3  2001  Nevada  2.4
4  2002  Nevada  2.9
5  2003  Nevada  3.2
```

In [ ]:
```
# If you pass a column that isn't contained in the dict, it will appear with
miss
# in the result
# Pass a column 'debt' that is not in the dictionary
# Fixed index to match the 6 rows of data
frame2 = pd.DataFrame(data, columns=['year', 'state', 'pop', 'debt'], index=
['one', 'two', 'three', 'four', 'five', 'six'])
print(frame2)
```

```
       year   state  pop debt
one    2000    Ohio  1.5  NaN
two    2001    Ohio  1.7  NaN
three  2002    Ohio  3.6  NaN
four   2001  Nevada  2.4  NaN
five   2002  Nevada  2.9  NaN
six    2003  Nevada  3.2  NaN
```

In [ ]:
```
#display a single column from frame2
print(frame2['year'])

# Retrieve a column as a Series using dict-like notation
print("\n", frame2['state'])

# Retrieve a column as a Series using attribute notation
print("\n", frame2.state)
```

```
one      2000
two      2001
three    2002
four     2001
five     2002
six      2003
Name: year, dtype: int64

 one        Ohio
two        Ohio
three      Ohio
four     Nevada
five     Nevada
six      Nevada
Name: state, dtype: object

 one        Ohio
two        Ohio
three      Ohio
four     Nevada
five     Nevada
six      Nevada
Name: state, dtype: object
```

In [ ]:
```
#Rows can also be retrieved by position or name with the special loc
attribute

# Retrieve the row at position 0
print(frame2. loc['one'])

# Retrieve the rows at positions 1 and 2
print(frame2.loc[['two', 'three']])
```

```
year     2000
state    Ohio
pop       1.5
debt      NaN
Name: one, dtype: object
       year state  pop debt
two    2001  Ohio  1.7  NaN
three  2002  Ohio  3.6  NaN
```

In [ ]:
```
# broadcast
frame2['debt'] = 12.4
print(frame2)
```

```
       year   state  pop  debt
one    2000    Ohio  1.5  12.4
two    2001    Ohio  1.7  12.4
three  2002    Ohio  3.6  12.4
four   2001  Nevada  2.4  12.4
five   2002  Nevada  2.9  12.4
six    2003  Nevada  3.2  12.4
```

In [ ]:
```
# or use arange() method -- > np.arange(6.)
import numpy as np
frame2['debt'] = np.arange(6.)
print(frame2)
```

```
       year   state  pop  debt
one    2000    Ohio  1.5   0.0
two    2001    Ohio  1.7   1.0
three  2002    Ohio  3.6   2.0
four   2001  Nevada  2.4   3.0
five   2002  Nevada  2.9   4.0
six    2003  Nevada  3.2   5.0
```

In [ ]:
```
val = pd.Series([-1.2, -1.5, -1.7], index=['two', 'four', 'five'])
frame2['debt'] = val
print(frame2)
```

```
       year   state  pop  debt
one    2000    Ohio  1.5   NaN
two    2001    Ohio  1.7  -1.2
three  2002    Ohio  3.6   NaN
four   2001  Nevada  2.4  -1.5
five   2002  Nevada  2.9  -1.7
six    2003  Nevada  3.2   NaN
```

In [ ]:
```
#Assigning a column that doesn't exist will create a new column.
frame2['eastern'] = frame2.state == '0hio'
print(frame2)
```

```
       year   state  pop  debt  eastern
one    2000    Ohio  1.5   NaN    False
two    2001    Ohio  1.7  -1.2    False
three  2002    Ohio  3.6   NaN    False
four   2001  Nevada  2.4  -1.5    False
five   2002  Nevada  2.9  -1.7    False
six    2003  Nevada  3.2   NaN    False
```

In [ ]:
```
#The del keyword will delete columns as with a dict.
del frame2['eastern']
print(frame2)
```

```
       year   state  pop  debt
one    2000    Ohio  1.5   NaN
two    2001    Ohio  1.7  -1.2
three  2002    Ohio  3.6   NaN
four   2001  Nevada  2.4  -1.5
five   2002  Nevada  2.9  -1.7
six    2003  Nevada  3.2   NaN
```

In [ ]:
```
#Another common form of data is a nested dict of dicts:
pop = {'Nevada': {2001: 2.4, 2002: 2.9},
'Ohio': {2000: 1.5, 2001: 1.7, 2002: 3.6}}
frame3 = pd.DataFrame(pop)
print(frame3)
```

```
      Nevada  Ohio
2001     2.4   1.7
2002     2.9   3.6
2000     NaN   1.5
```

In [ ]:
```
#The keys in the inner dicts are combined and sorted to form the
#index in the result. This isn't true if an explicit index is specified:

frame3 = pd.DataFrame(pop, index=[2001, 2002, 2003])
print(frame3)
```

```
      Nevada  Ohio
2001     2.4   1.7
2002     2.9   3.6
2003     NaN   NaN
```

In [ ]:
```
# You can transpose the DataFrame (swap rows and columns) with similar
#syntax to a NumPy array

print(frame3.T)
```

```
        2001  2002  2003
Nevada   2.4   2.9   NaN
Ohio     1.7   3.6   NaN
```

In [ ]:
```
pdata = {'Ohio': frame3['Ohio' ] [ :- 1],
'Nevada': frame3[ 'Nevada' ] [ : 2]}
pd. DataFrame (pdata)+
```

Out[16]:

	Ohio	Nevada
2001	1.7	2.4
2002	3.6	2.9

In [ ]:

```python
# prompt: example for all above Index methods and properties
import pandas as pd
import numpy as np

# Sample Series
s = pd.Series(np.arange(5), index=['a', 'b', 'c', 'd', 'e'])
print(s)
# Index properties
print("Index:", s.index) # Display the index
print("Index values:", s.index.values) # Array of index values
#print("Is monotonic:", s.index.is_monotonic) # Check if index is monotonic
print("Is unique:", s.index.is_unique) # Check if index values are unique
# Index methods
print("Index of 'c':", s.index.get_loc('c')) # Get integer location of 'c'
print("Slice with 'b':'d':\n", s['b':'d']) # Slicing using index labels
print("Fancy indexing with ['a', 'e']:\n", s[['a', 'e']]) # Selecting
specific elements
```

```
a    0
b    1
c    2
d    3
e    4
dtype: int64
Index: Index(['a', 'b', 'c', 'd', 'e'], dtype='object')
Index values: ['a' 'b' 'c' 'd' 'e']
Is unique: True
Index of 'c': 2
Slice with 'b':'d':
 b    1
c    2
d    3
dtype: int64
Fancy indexing with ['a', 'e']:
 a    0
e    4
dtype: int64
```

In [ ]:

```
#If a DataFrame's index and columns have their name attributes set,
#these will also be displayed
data = np.arange(0, 9).reshape(3, 3)
index = ['Ohio', 'Colorado', 'New York']
columns = ['one', 'two', 'three']
df = pd.DataFrame(data, index=index, columns=columns)
df.index.name = 'state'
df.columns.name = 'number'
print(df,"\n")

df
```

```
number    one  two  three
state
Ohio        0    1      2
Colorado    3    4      5
New York    6    7      8
```

Out[5]:

number	one	two	three
state			
Ohio	0	1	2
Colorado	3	4	5
New York	6	7	8

In [ ]:
```python
# 9 - REINDEXING
# Gokul G (241801068)
# reindexing
import pandas as pd
import numpy as np
obj = pd.Series([4.5, 7.2, -5.3, 3.6], index=['d', 'b', 'a', 'c'])
print(obj)
obj2 = obj.reindex(['a', 'b', 'c', 'd', 'e'])
obj2
```

```
d    4.5
b    7.2
a   -5.3
c    3.6
dtype: float64
```

Out[6]:

	0
a	-5.3
b	7.2
c	3.6
d	4.5
e	NaN

dtype: float64

In [ ]:
```python
# #Interpolation or filling
obj3 = pd.Series(['blue', 'purple', 'yellow'], index=[0, 2, 4])
print(obj3)
obj4 = obj3.reindex(range(6), method='ffill') #forward-fills the values
obj4
```

```
0      blue
2    purple
4    yellow
dtype: object
```

Out[7]:

	0
0	blue
1	blue
2	purple
3	purple
4	yellow
5	yellow

dtype: object

In [ ]:

```
# reindex can alter either the (row) index, columns, or both some other
example

import pandas as pd
import numpy as np

# Reindexing rows
obj = pd.Series([4.5, 7.2, -5.3, 3.6], index=['d', 'b', 'a', 'c'])
obj2 = obj.reindex(['a', 'b', 'c', 'd', 'e'])
print("Reindexing rows:")
print(obj2)

# Reindexing columns
df= pd.DataFrame(np.arange(9).reshape((3, 3)),
index=['a', 'c', 'd'],
columns=['Ohio', 'Texas', 'California'])
df2 = df.reindex(columns=['Texas', 'Utah', 'California'])
print("\nReindexing columns:")
print(df2)

#The columns can be reindexed with the columns keyword:
# Reindexing both rows and columns (fixed the unclosed bracket/parenthesis
syntax error)
df3 = df.reindex(index=['a', 'b', 'c', 'd'], columns=["Dallas", 'Texas',
'Utah', 'California'])
print("\nReindexing both rows and columns:")
print(df3)
```

```
Reindexing rows:
a   -5.3
b    7.2
c    3.6
d    4.5
e    NaN
dtype: float64

Reindexing columns:
   Texas  Utah  California
a      1   NaN           2
c      4   NaN           5
d      7   NaN           8

Reindexing both rows and columns:
   Dallas  Texas  Utah  California
a     NaN    1.0   NaN         2.0
b     NaN    NaN   NaN         NaN
c     NaN    4.0   NaN         5.0
d     NaN    7.0   NaN         8.0
```

In [ ]:

```
# method
# 'ffill' or 'pad': Fill values forward
obj3= pd.Series(['blue', 'purple', 'yellow'], index=[0, 2, 4])
obj4 = obj3.reindex(range(6), method='ffill')
print("Method 'ffill':")
print(obj4)

# 'bfill' or 'backfill': Fill values backward
obj5 = obj3.reindex(range(6), method='bfill')
print("\nMethod 'bfill':")
print(obj5)

# fill_value
# Fill missing values with a specific value
obj6 = obj3.reindex(range(6), fill_value='red')
print("\nFill value:")
print(obj6)

# limit
# Limit the number of consecutive filled values
obj7 = pd.Series([1,2,3], index=[0, 2, 5])
obj8 = obj7.reindex(range(7), method='ffill', limit=1)
print("\nLimit:")
print(obj8)
```

```
Method 'ffill':
0      blue
1      blue
2    purple
3    purple
4    yellow
5    yellow
dtype: object

Method 'bfill':
0      blue
1    purple
2    purple
3    yellow
4    yellow
5       NaN
dtype: object

Fill value:
0      blue
1       red
2    purple
3       red
4    yellow
5       red
dtype: object

Limit:
```

```
0    1.0
1    1.0
2    2.0
3    2.0
4    NaN
5    3.0
6    3.0
dtype: float64
```

In [ ]:
```
# level, multiindex
# Used when reindexing a MultiIndex DataFrame
arrays = [['bar', 'bar', 'baz', 'baz', 'foo', 'foo', 'qux', 'qux'],
['one', 'two', 'one', 'two', 'one', 'two', 'one', 'two']]
tuples = list(zip(*arrays))
index = pd.MultiIndex. from_tuples(tuples, names=['first', 'second'])
df = pd.DataFrame(np.random.randn(8, 2), index=index, columns=['A', 'B'])
df.reindex(['bar', 'baz', 'foo', 'qux'], level='first')
print("\nLevel:")
print(df)
```

```
Level:
                     A         B
first second
bar   one     1.683586  0.801730
      two     1.306419  0.999321
baz   one    -2.026799 -0.624934
      two    -1.244467  0.050019
foo   one    -0.505641  0.956451
      two     0.336214  1.557455
qux   one    -0.071013  0.033616
      two    -0.938291 -0.388092
```

In [ ]:
```
# Dropping Entries from an Axis
obj = pd.Series(np.arange(5.), index=['a', 'b', 'c', 'd', 'e'])
new_obj = obj.drop('c')
print(new_obj)
obj.drop(['d', 'c'])
```

```
a    0.0
b    1.0
d    3.0
e    4.0
dtype: float64
```

Out[13]:

	0
a	0.0
b	1.0
e	4.0

dtype: float64

In [ ]:
```
#With DataFrame, index values can be deleted from either axis.
data = pd.DataFrame(np.arange(16).reshape((4, 4)),
index=['Ohio', 'Dallas', 'NJ', 'New York'],
columns=['one', 'two', 'three', 'four' ])
data
```

Out[14]:

	one	two	three	four
Ohio	0	1	2	3
Dallas	4	5	6	7
NJ	8	9	10	11
New York	12	13	14	15

In [ ]:
```
#Calling drop with a sequence of labels will drop values from the row labels (axi
data.drop(['NJ', 'Ohio'])
```

Out[19]:

	one	two	three	four
Dallas	4	5	6	7
New York	12	13	14	15

In [ ]:
```
# You can drop values from the columns by passing axis=1 or axis='columns':
data.drop('three', axis=1)
```

Out[20]:

	one	two	four
Ohio	0	1	3
Dallas	4	5	7
NJ	8	9	11
New York	12	13	15

In [ ]:
```
data.drop(['one', 'four' ], axis='columns')
```

Out[22]:

	two	three
Ohio	1	2
Dallas	5	6
NJ	9	10
New York	13	14

In [ ]:
```
data
```

Out[23]:

	one	two	three	four
Ohio	0	1	2	3
Dallas	4	5	6	7
NJ	8	9	10	11
New York	12	13	14	15

In [ ]:
```
#inplace=true show example with data
# Dropping rows with inplace=True
data.drop(['NJ', 'Ohio'], inplace=True)
print(data)
```

```
          one  two  three  four
Dallas      4    5      6     7
New York   12   13     14    15
```

In [ ]:
```
data
```

Out[25]:

	one	two	three	four
Dallas	4	5	6	7
New York	12	13	14	15

In [ ]:

```
# prompt: series indexing
obj = pd.Series(np.arange(4.), index=['a', 'b', 'c', 'd'])
print("obj['b']=\n",obj ['b'])
print("obj [1]=\n", obj [1])
print("obj[2:4]= \n", obj [2:4] )
print("obj [['b', 'a', 'd']]= \n", obj[['b', 'a', 'd']])
print("obj [b: d]=\n", obj ['b':'d']) #no n-1 concept
print("obj[[1, 3]]=\n",obj[[1, 3]])
print("obj[obj < 2]= \n", obj [obj < 2])
```

```
obj['b']=
 1.0
obj [1]=
 1.0
obj[2:4]=
 c    2.0
d    3.0
dtype: float64
obj [['b', 'a', 'd']]=
 b    1.0
a    0.0
d    3.0
dtype: float64
obj [b: d]=
 b    1.0
c    2.0
d    3.0
dtype: float64
obj[[1, 3]]=
 b    1.0
d    3.0
dtype: float64
obj[obj < 2]=
 a    0.0
b    1.0
dtype: float64
```

```
/tmp/ipykernel_1916/776473932.py:4: FutureWarning: Series.__getitem__
treating keys as positions is deprecated. In a future version, integer keys
will always be treated as labels (consistent with DataFrame behavior). To
access a value by position, use `ser.iloc[pos]`
  print("obj [1]=\n", obj [1])
/tmp/ipykernel_1916/776473932.py:8: FutureWarning: Series.__getitem__
treating keys as positions is deprecated. In a future version, integer keys
will always be treated as labels (consistent with DataFrame behavior). To
access a value by position, use `ser.iloc[pos]`
  print("obj[[1, 3]]=\n",obj[[1, 3]])
```

In [ ]:
```
data = pd.DataFrame(np.arange(24).reshape((6, 4)),
index=['Ohio', 'Dallas', 'Boston', 'NJ', 'New York', 'Austin'],
columns=['one', 'two', 'three', 'four' ])

data
```

Out[28]:

	one	two	three	four
Ohio	0	1	2	3
Dallas	4	5	6	7
Boston	8	9	10	11
NJ	12	13	14	15
New York	16	17	18	19
Austin	20	21	22	23

In [ ]:
```
# Selecting a single column, which yields a Series, equivalent to df.
data['two']
```

Out[29]:

	two
Ohio	1
Dallas	5
Boston	9
NJ	13
New York	17
Austin	21

dtype: int64

In [ ]:
```
# Selecting via [], which slices the rows.
data [1:5] #n-1 concept
```

Out[30]:

	one	two	three	four
Dallas	4	5	6	7
Boston	8	9	10	11
NJ	12	13	14	15
New York	16	17	18	19

In [ ]:
```
# Selecting on a multi-axis by label
data. loc['Dallas', ['two', 'three']]
```

Out[31]:

	Dallas
two	5
three	6

dtype: int64

In [ ]:
```
# Showing label slicing, both endpoints are included
data. loc['Dallas': 'New York', 'two']
```

Out[32]:

	two
Dallas	5
Boston	9
NJ	13
New York	17

dtype: int64

In [ ]:
```
# Reduction in the dimensions of the returned object
data. loc['Dallas', ['two']]
```

Out[36]:

	Dallas
two	5

dtype: int64

In [ ]:
```
# For getting a scalar value
data. loc['Dallas', 'two']
```

Out[37]:
```
np.int64(5)
```

In [ ]:
```
# For getting fast access to a scalar (equiv to the prior method)
data.at ['Dallas', 'one']
```

Out[38]:
```
np.int64(4)
```

In [ ]:
```
#'loc' function is mainly used when we want to select rows and columns based
on t
#'iloc' is used when choosing rows and columns based on specific positions.
# Select via the position of the passed integers
data.iloc[3]
```

Out[39]:

	NJ
one	12
two	13
three	14
four	15

dtype: int64

In [ ]:
```
# By integer slices, acting similar to numpy/python
data.iloc[2:5, 0:2]
```

Out[40]:

	one	two
Boston	8	9
NJ	12	13
New York	16	17

In [ ]:
```
# By lists of integer position locations, similar to the numpy/python style
data.iloc[[1, 3, 5], [0, 3]]
```

Out[41]:

	one	four
Dallas	4	7
NJ	12	15
Austin	20	23

In [ ]:
```
# For slicing rows explicitly
data.iloc[1:4, :]
```

Out[42]:

	one	two	three	four
Dallas	4	5	6	7
Boston	8	9	10	11
NJ	12	13	14	15

In [ ]:
```
# For slicing columns explicitly
data.iloc[:, 1:3]
```

Out[43]:

	two	three
Ohio	1	2
Dallas	5	6
Boston	9	10
NJ	13	14
New York	17	18
Austin	21	22

In [ ]:
```
# For getting a value explicitly
data.iloc[1, 1]
```

Out[44]:
```
np.int64(5)
```

In [ ]:
```
# For getting fast access to a scalar (equiv to the prior method)
data.iat [1, 1]
```

Out[45]:
```
np.int64(5)
```

In [ ]:
```
# Using a single column's values to select data.
print(data.three > 5)
data[data.three > 5]
```

```
Ohio        False
Dallas       True
Boston       True
NJ           True
New York     True
Austin       True
Name: three, dtype: bool
```

Out[46]:

	one	two	three	four
Dallas	4	5	6	7
Boston	8	9	10	11
NJ	12	13	14	15
New York	16	17	18	19
Austin	20	21	22	23

In [ ]:

```
# A boolean DataFrame
data < 5
data[data < 5] = 0
data
```

Out[47]:

	one	two	three	four
Ohio	0	0	0	0
Dallas	0	5	6	7
Boston	8	9	10	11
NJ	12	13	14	15
New York	16	17	18	19
Austin	20	21	22	23

In [ ]:

```python
# 10 - Arithmetic and Data Alignment
# Gokul G (241801068)
# 5 subjects marks as separate series, find total marks for eachstudent,
mention subject number also

import pandas as pd

# Sample marks for 5 subjects and 7 students
students = [f'Student {i}' for i in range(1, 8)]
subject1 = pd.Series([80, 75, 92, 68, 70, 95, 88], index=students)
subject2 = pd.Series([72, 88, 79, 91, 65, 82, 76], index=students)
subject3 = pd.Series([90, 68, 85, 73, 78, 92, 80], index=students)
subject4 = pd.Series([65, 92, 82, 88, 75, 79, 85], index=students)
subject5 = pd.Series([78, 70, 95, 62, 83, 86, 91], index=students)

# Calculate total marks for each student
total_marks = subject1 + subject2 + subject3 + subject4 + subject5
print("Total marks= \n", total_marks)
# Print total marks with subject numbers
print("Total Marks for Each Student:")
for i, (student, marks) in enumerate(total_marks.items()):
    print(f"Student {i+1} ({student}): {marks}")
```

```
Total marks=
 Student 1    385
Student 2    393
Student 3    433
Student 4    382
Student 5    371
Student 6    434
Student 7    420
dtype: int64
Total Marks for Each Student:
Student 1 (Student 1): 385
Student 2 (Student 2): 393
Student 3 (Student 3): 433
Student 4 (Student 4): 382
Student 5 (Student 5): 371
Student 6 (Student 6): 434
Student 7 (Student 7): 420
```

In [ ]:

```python
# dataframe arithmetic operators

import pandas as pd

# Sample DataFrames
df1 = pd.DataFrame({'A': [1, 2, 3], 'B': [4, 5, 6]})
df2 = pd.DataFrame({'A': [7, 8,9],'B': [10, 11,12]})
print(df1)
print(df2)

# Arithmetic operators on DataFrames
# Addition
print("Addition:\n", df1 + df2)

# Subtraction
print("\nSubtraction:\n", df1 - df2)

# Multiplication
print("\nMultiplication:\n", df1 * df2)

# Division
print("\nDivision:\n", df1 / df2)
```

```
   A  B
0  1  4
1  2  5
2  3  6
   A   B
0  7  10
1  8  11
2  9  12
Addition:
     A   B
0   8  14
1  10  16
2  12  18

Subtraction:
    A  B
0 -6 -6
1 -6 -6
2 -6 -6

Multiplication:
     A   B
0   7  40
1  16  55
2  27  72

Division:
           A         B
0  0.142857  0.400000
```

```
1  0.250000  0.454545
2  0.333333  0.500000
```

In [ ]:
```python
#DataFrame, alignment is performed on both the rows and the columns

import pandas as pd

# Sample DataFrames with different indices and columns
df3 = pd.DataFrame({'A': [1, 2, 3], 'B': [4, 5, 6]}, index=[0, 1, 2])
df4 = pd.DataFrame({'B': [7, 8, 9], 'C': [10, 11, 12]}, index=[1, 2, 3])
print(df3)
print(df4)

# Arithmetic operation with alignment
result = df3 + df4
print("Result of df3 + df4 with alignment:\n", result)
```

```
   A  B
0  1  4
1  2  5
2  3  6
   B   C
1  7  10
2  8  11
3  9  12
Result of df3 + df4 with alignment:
     A     B   C
0 NaN   NaN NaN
1 NaN  12.0 NaN
2 NaN  14.0 NaN
3 NaN   NaN NaN
```

In [ ]:
```python
df3 - df4
```

Out[6]:

	A	B	C
0	NaN	NaN	NaN
1	NaN	-2.0	NaN
2	NaN	-2.0	NaN
3	NaN	NaN	NaN

In [ ]:
```python
# add, radd Methods for addition (+)
result_add = df3.add(df4, fill_value=0)
print("\nResult of df3.add(df4) with fill_value=0:\n", result_add)
```

```
Result of df3.add(df4) with fill_value=0:
      A     B     C
0  1.0   4.0   NaN
1  2.0  12.0  10.0
2  3.0  14.0  11.0
3  NaN   9.0  12.0
```

In [ ]:

```
result_radd = df3.radd(df4, fill_value=0)
print("\nResult of df3. radd(df4) with fill_value=0:\n", result_radd)

 # sub, rsub Methods for subtraction (-)
result_sub = df3.sub(df4, fill_value=0)
print("\nResult of df3.sub(df4) with fill_value=0:\n", result_sub)

result_rsub = df3.rsub(df4, fill_value=0)
print("\nResult of df3.rsub(df4) with fill_value=0:\n", result_rsub)
```

```
Result of df3. radd(df4) with fill_value=0:
      A     B     C
0  1.0   4.0   NaN
1  2.0  12.0  10.0
2  3.0  14.0  11.0
3  NaN   9.0  12.0

Result of df3.sub(df4) with fill_value=0:
      A    B     C
0  1.0  4.0   NaN
1  2.0 -2.0 -10.0
2  3.0 -2.0 -11.0
3  NaN -9.0 -12.0

Result of df3.rsub(df4) with fill_value=0:
      A    B     C
0 -1.0 -4.0   NaN
1 -2.0  2.0  10.0
2 -3.0  2.0  11.0
3  NaN  9.0  12.0
```

In [ ]:

```
# div, rdiv Methods for division (/)
result_div = df3.div(df4, fill_value=1) # Avoid division by zero
print("\nResult of df3.div(df4) with fill_value=1:\n", result_div)

result_rdiv = df3.rdiv(df4, fill_value=1) # Avoid division by zero
print("\nResult of df3.rdiv(df4) with fill_value=1:\n", result_rdiv)

# floordiv, rfloordiv Methods for floor division (//)
result_floordiv = df3.floordiv(df4, fill_value=1) # Avoid division by zero
print("\nResult of df3. floordiv(df4) with fill_value=1:\n",
result_floordiv)

result_rfloordiv = df3.rfloordiv(df4, fill_value=1) # Avoid division by zero
print("\nResult of df3.rfloordiv(df4) with fill_value=1:\n",
result_rfloordiv)
```

```
Result of df3.div(df4) with fill_value=1:
      A         B         C
0  1.0  4.000000       NaN
1  2.0  0.714286  0.100000
2  3.0  0.750000  0.090909
3  NaN  0.111111  0.083333

Result of df3.rdiv(df4) with fill_value=1:
           A         B     C
0  1.000000  0.250000   NaN
1  0.500000  1.400000  10.0
2  0.333333  1.333333  11.0
3       NaN  9.000000  12.0

Result of df3. floordiv(df4) with fill_value=1:
      A    B    C
0  1.0  4.0  NaN
1  2.0  0.0  0.0
2  3.0  0.0  0.0
3  NaN  0.0  0.0

Result of df3.rfloordiv(df4) with fill_value=1:
      A    B     C
0  1.0  0.0   NaN
1  0.0  1.0  10.0
2  0.0  1.0  11.0
3  NaN  9.0  12.0
```

In [ ]:

```
# mul, rmul Methods for multiplication (*)
result_mul = df3.mul(df4, fill_value=1)
print("\nResult of df3.mul(df4) with fill_value=1:\n", result_mul)
result_rmul = df3.rmul(df4, fill_value=1)
print("\nResult of df3.rmul(df4) with fill_value=1:\n", result_rmul)
```

```
Result of df3.mul(df4) with fill_value=1:
      A     B     C
0  1.0   4.0   NaN
1  2.0  35.0  10.0
2  3.0  48.0  11.0
3  NaN   9.0  12.0

Result of df3.rmul(df4) with fill_value=1:
      A     B     C
0  1.0   4.0   NaN
1  2.0  35.0  10.0
2  3.0  48.0  11.0
3  NaN   9.0  12.0
```

In [ ]:

```
# pow, rpow for series and dataframe
# Series
series1 = pd.Series([1, 2, 3], index=['a', 'b', 'c'])
series2 = pd.Series([4, 5, 6], index=['b', 'c', 'd'])

result_pow_series = series1.pow(series2, fill_value=1)
print("\nResult of series1.pow(series2) with fill_value=1:\n",
result_pow_series)

result_rpow_series = series1.rpow(series2, fill_value=1)
print("\nResult of series1.rpow(series2) with fill_value=1:\n",
result_rpow_series)

# DataFrame
result_pow_df = df3.pow(df4, fill_value=1)
print("\nResult of df3.pow(df4) with fill_value=1:\n", result_pow_df)

result_rpow_df = df3.rpow(df4, fill_value=1)
print("\nResult of df3.rpow(df4) with fill_value=1:\n", result_rpow_df)
```

```
Result of series1.pow(series2) with fill_value=1:
 a      1.0
b     16.0
c    243.0
d      1.0
dtype: float64

Result of series1.rpow(series2) with fill_value=1:
 a      1.0
b     16.0
c    125.0
d      6.0
dtype: float64

Result of df3.pow(df4) with fill_value=1:
      A          B    C
0  1.0        4.0  NaN
1  2.0    78125.0  1.0
2  3.0  1679616.0  1.0
3  NaN        1.0  1.0

Result of df3.rpow(df4) with fill_value=1:
      A         B     C
0  1.0       1.0   NaN
1  1.0   16807.0  10.0
2  1.0  262144.0  11.0
3  NaN       9.0  12.0
```

In [ ]:

```
# dataframe broadcast like df=2

import pandas as pd
df = pd.DataFrame({'A': [1, 2, 3], 'B': [4, 5, 6]})
df = df + 2
print(df)
print(df)

import numpy as np
frame = pd.DataFrame(np.random.randn(4, 3), columns=list('bde'),
index=['Utah', 'Ohio', 'Texas', 'Oregon'])
print(frame)
print(np.abs(frame))
```

```
   A  B
0  3  6
1  4  7
2  5  8
   A  B
0  3  6
1  4  7
2  5  8
               b         d         e
Utah    0.739909  1.272676 -0.112087
Ohio    1.380817 -1.279391  1.168967
Texas  -0.156326 -1.454323  0.106219
Oregon -0.217817 -0.124834  1.297059
               b         d         e
Utah    0.739909  1.272676  0.112087
Ohio    1.380817  1.279391  1.168967
Texas   0.156326  1.454323  0.106219
Oregon  0.217817  0.124834  1.297059
```

In [ ]:

```
#apply
f = lambda x: x.max() - x.min()
df_apply = frame.apply(f)
print(df_apply)
df_apply_axis = frame.apply(f, axis='columns') #apply lambda function to
columns
print(df_apply_axis)

#display min, max value for each column
def f(x):
  return pd.Series([x.min(), x.max()], index=['min', 'max'])

df_apply_series = frame.apply(f)
print(df_apply_series)

# Map function
format = lambda x: '%.2f' % x
df_map = frame['e'].map(format)
print(df_map)
```

```
b    1.598634
d    2.726999
e    1.409146
dtype: float64
Utah      1.384763
Ohio      2.660208
Texas     1.560542
Oregon    1.514876
dtype: float64
            b         d         e
min -0.217817 -1.454323 -0.112087
max  1.380817  1.272676  1.297059
Utah      -0.11
Ohio       1.17
Texas      0.11
Oregon     1.30
Name: e, dtype: object
```

In [ ]:

```
#sort series and dataframe

# Sorting a series
obj = pd.Series(range(4), index=['d', 'a', 'b', 'c'])
df_sort_series = obj.sort_index()
print(df_sort_series)

# Sorting a dataframe
frame = pd.DataFrame(np.arange(8).reshape((2, 4)),
index=['three', 'one'],
columns=['d', 'a', 'b', 'c'])
df_sort_dataframe = frame.sort_index()#sort row name-wise
print("I \n",df_sort_dataframe)
df_sort_dataframe_axis1 = frame.sort_index(axis=1)#sort all columns in order
print("II \n",df_sort_dataframe_axis1)
df_sort_dataframe_ascending = frame.sort_index(axis=1, ascending=False)
print("III \n",df_sort_dataframe_ascending)
df_sort_values_by_column = frame.sort_values(by='b') #sort b and extend the
sort
print("IV \n", df_sort_values_by_column)
```

```
a    1
b    2
c    3
d    0
dtype: int64
I
        d  a  b  c
one     4  5  6  7
three   0  1  2  3
II
        a  b  c  d
three   1  2  3  0
one     5  6  7  4
III
        d  c  b  a
three   0  3  2  1
one     4  7  6  5
IV
        d  a  b  c
three   0  1  2  3
one     4  5  6  7
```

In [ ]:
```
#Any missing values are sorted to the end of the Series by default

obj = pd.Series([4, np.nan, 7, np.nan, -3, 2])
df_sort_series_na = obj.sort_values()
print(df_sort_series_na)
```

```
4   -3.0
5    2.0
0    4.0
2    7.0
1    NaN
3    NaN
dtype: float64
```

In [ ]:
```
# When sorting a DataFrame, you can use the data in one or more columns as
the sort
# keys.

import pandas as pd
import numpy as np
frame = pd.DataFrame({'b': [4, 7, -3, 2], 'a': [0, 1, 0, 1]})
print("I \n",frame)
df_sort_dataframe_by_column = frame. sort_values(by='b')
print("II \n", df_sort_dataframe_by_column)
df_sort_dataframe_by_multiple_columns = frame.sort_values(by=['a', 'b'])
print("III \n",df_sort_dataframe_by_multiple_columns)
```

```
I
    b  a
0  4  0
1  7  1
2 -3  0
3  2  1
II
    b  a
2 -3  0
3  2  1
0  4  0
1  7  1
III
    b  a
2 -3  0
0  4  0
3  2  1
1  7  1
```

In [ ]:

```
#ranking
# Ranking assigns ranks from one through the number of valid data points in
an array.
obj = pd.Series([7, -5, 7, 3, 2, 0, 4])
df_rank = obj.rank()
print("Rank \n", df_rank)
df_rank_method_first = obj.rank(method='first') #ranks assigned in order
they appear in the data
print("rank- Method=first \n", df_rank_method_first)

#ranking in descending order
df_rank_ascending = obj.rank(ascending=False, method='max')
print("rank- Method=max \n", df_rank_ascending)

#DataFrame can compute ranks over the rows or the columns
frame = pd.DataFrame({'b': [4.3, 7, -3, 2], 'a': [0, 1, 0, 1],
'c': [-2, 5, 8, -2.5]})
print("Rank over rows \n", frame)
df_rank_dataframe = frame. rank (axis='columns' )
print("Rank over columns \n", df_rank_dataframe)
```

```
Rank
 0    6.5
1    1.0
2    6.5
3    4.0
4    3.0
5    2.0
6    5.0
dtype: float64
rank- Method=first
 0    6.0
1    1.0
2    7.0
3    4.0
4    3.0
5    2.0
6    5.0
dtype: float64
rank- Method=max
 0    2.0
1    7.0
2    2.0
3    4.0
4    5.0
5    6.0
6    3.0
dtype: float64
Rank over rows
      b  a    c
0  4.3  0 -2.0
1  7.0  1  5.0
2 -3.0  0  8.0
```

```
3  2.0  1 -2.5
Rank over columns
      b    a    c
0  3.0  2.0  1.0
1  3.0  1.0  2.0
2  1.0  2.0  3.0
3  3.0  2.0  1.0
```

In [ ]:

```python
#is_unique property

import pandas as pd
import numpy as np
obj = pd.Series(range(5), index=['a', 'a', 'b', 'b', 'c'])
print(obj.index.is_unique)

print()

# DataFrame is_unique
obj = pd.Series(range(5), index=['a', 'a', 'b', 'b', 'c'])
print(obj.index.is_unique)
df = pd.DataFrame(np.random.randn(4, 3), index=['a', 'a', 'b', 'b'])
print(df.index.is_unique)

# Another example

df2 = pd.DataFrame({'A': [1, 2, 3], 'B': [4, 5, 6]}, index=['x', 'y', 'z'])
print(df2. index.is_unique)
```

```
False

False
False
True
```

In [ ]:
```
# 5 subjects marks as dataframe, find total marks for each student, mention subject number also

import pandas as pd
import numpy as np

# Sample data for 5 subjects and a few students
data = {'Math': np.random.randint(50, 100, 5),
'Science': np.random.randint(50, 100, 5),
'English': np.random.randint(50, 100, 5),
'History': np.random.randint(50, 100, 5),
'Geography': np.random.randint(50, 100, 5)}

df = pd.DataFrame(data, index=['Student1', 'Student2', 'Student3', 'Student4', 'Student5'])
df
```

Out[22]:

	Math	Science	English	History	Geography
Student1	88	68	98	54	83
Student2	75	90	60	89	87
Student3	89	96	54	89	94
Student4	71	61	86	70	73
Student5	97	91	74	65	94

In [ ]:
```
# Calculate total marks for each student
df['Total'] = df.sum(axis=1)
df
```

Out[26]:

	Math	Science	English	History	Geography	Total	Subject Count
Student1	88	68	98	54	83	1573	5
Student2	75	90	60	89	87	1613	5
Student3	89	96	54	89	94	1697	5
Student4	71	61	86	70	73	1453	5
Student5	97	91	74	65	94	1693	5

In [ ]:
```
df.rank()
```

Out[27]:

	Math	Science	English	History	Geography	Total	Subject Count
Student1	3.0	2.0	5.0	1.0	2.0	2.0	3.0
Student2	2.0	3.0	2.0	4.5	3.0	3.0	3.0
Student3	4.0	5.0	1.0	4.5	4.5	5.0	3.0
Student4	1.0	1.0	4.0	3.0	1.0	1.0	3.0
Student5	5.0	4.0	3.0	2.0	4.5	4.0	3.0

In [ ]:
```
df.rank(ascending=False)
```

Out[29]:

	Math	Science	English	History	Geography	Total	Subject Count
Student1	3.0	4.0	1.0	5.0	4.0	4.0	3.0
Student2	4.0	3.0	4.0	1.5	3.0	3.0	3.0
Student3	2.0	1.0	5.0	1.5	1.5	1.0	3.0
Student4	5.0	5.0	2.0	3.0	5.0	5.0	3.0
Student5	1.0	2.0	3.0	4.0	1.5	2.0	3.0

In [ ]:
```
# Add a column for subject count
df['Subject Count'] = len(df. columns) - 2 # Subtract 'Total' and 'Subject Count' columns
print(df)
```

```
          Math  Science  English  History  Geography  Total  Subject Count
Student1    88       68       98       54         83   1573              5
Student2    75       90       60       89         87   1613              5
Student3    89       96       54       89         94   1697              5
Student4    71       61       86       70         73   1453              5
Student5    97       91       74       65         94   1693              5
```

In [ ]:
```
# 11 - Summarizing and Computing Descriptive Statistics
# Gokul G (241801068)
# generate a data frame
import pandas as pd
import numpy as np
df = pd.DataFrame({'Column1': [1, 2, np.nan, 4],
'Column2': [5, np.nan, 7, 8]})
print(df)
```

```
   Column1  Column2
0      1.0      5.0
1      2.0      NaN
2      NaN      7.0
3      4.0      8.0
```

In [ ]:
```
#sum
df.sum()
```

Out[35]:

	0
Column1	7.0
Column2	20.0

dtype: float64

In [ ]:
```
df.sum(axis=1)
```

Out[37]:

	0
0	6.0
1	2.0
2	7.0
3	12.0

dtype: float64

In [ ]:
```
df.mean()
```

Out[39]:

	0
Column1	2.333333
Column2	6.666667

dtype: float64

In [ ]:
```
print(df)
df.mean(axis='columns')
```

```
   Column1  Column2
0      1.0      5.0
1      2.0      NaN
2      NaN      7.0
3      4.0      8.0
```

Out[40]:

	0
0	3.0
1	2.0
2	7.0
3	6.0

dtype: float64

In [ ]:
```
df.mean(axis='columns', skipna=False)
```

Out[42]:

	0
0	3.0
1	NaN
2	NaN
3	6.0

dtype: float64

In [ ]:
```
#idxmin idxmax - Minimum and minimum value's index

min_index = df.idxmin()
max_index = df.idxmax()
print("Minimum indices:", min_index)
print("Maximum indices:", max_index)
```

```
Minimum indices: Column1    0
Column2    0
dtype: int64
Maximum indices: Column1    3
Column2    3
dtype: int64
```

In [ ]:
```
# Sample DataFrame (replace with your actual data)
df = pd.DataFrame({'A': [1, 2,3, 4, 5],
'B': [5, 4, 3, 2, 1]})

# Calculate cumulative sum for each column
cumsum_df = df.cumsum()
print("Cumulative sum:\n", cumsum_df)
```

```
Cumulative sum:
     A   B
0   1   5
1   3   9
2   6  12
3  10  14
4  15  15
```

In [ ]:
```
df.describe()
```

Out[45]:

	A	B
count	5.000000	5.000000
mean	3.000000	3.000000
std	1.581139	1.581139
min	1.000000	1.000000
25%	2.000000	2.000000
50%	3.000000	3.000000
75%	4.000000	4.000000
max	5.000000	5.000000

In [ ]:
```
# Summary on non-numeric DataFrame
df_non_numeric = pd.DataFrame({
'Name': ['Alice', 'Bob', 'Charlie', 'Bob'],
'City': ['New York', 'Los Angeles', 'Chicago', 'Alberta'],
'Country': ['USA', 'USA', 'USA', 'Canada']
})

# Summary of non-numeric data
summary = df_non_numeric.describe(include='object')
print(summary)
```

```
        Name      City Country
count      4         4       4
unique     3         4       2
top      Bob  New York     USA
freq       2         1       3
```

In [ ]:

```python
#Correlation and Covariance in dataframe

import pandas as pd
import numpy as np

# Sample DataFrame
data = {'A': [1, 2, 3, 4, 5],
'B': [5, 4, 3, 2, 1],
'C': [2, 3, 5, 1, 4]}
df = pd.DataFrame(data)
print(df)

# Calculate correlation matrix
correlation_matrix = df.corr()
print("Correlation Matrix:\n", correlation_matrix)

# Calculate covariance matrix
covariance_matrix = df.cov()
print("\nCovariance Matrix:\n", covariance_matrix)
```

```
   A  B  C
0  1  5  2
1  2  4  3
2  3  3  5
3  4  2  1
4  5  1  4
Correlation Matrix:
      A    B    C
A  1.0 -1.0  0.2
B -1.0  1.0 -0.2
C  0.2 -0.2  1.0

Covariance Matrix:
      A    B    C
A  2.5 -2.5  0.5
B -2.5  2.5 -0.5
C  0.5 -0.5  2.5
```

In [ ]:

```python
import pandas as pd
import numpy as np

# Sample DataFrame
data = {'A': [1, 2, 3, 4, 5],
'B': [5, 4, 3, 2, 1],
'C': [2, 3, 5, 1, 4]}
df = pd.DataFrame(data)

print(df)

# #Unique Values, Value Counts, and Membership
# Get unique values in column 'A'
unique_values = df['A'].unique()
print("\nUnique values in column 'A':", unique_values)

# Count occurrences of each value in column 'B'
value_counts = df['B'].value_counts()
print("\nValue counts in column 'B':\n", value_counts)

# Check if value 3 is present in column 'C'
is_present = 3 in df['C'].values
print("\nIs 3 present in column 'C' ?: ", is_present)
```

```
   A  B  C
0  1  5  2
1  2  4  3
2  3  3  5
3  4  2  1
4  5  1  4

Unique values in column 'A': [1 2 3 4 5]

Value counts in column 'B':
 B
5    1
4    1
3    1
2    1
1    1
Name: count, dtype: int64

Is 3 present in column 'C' ?:  True
```

In [ ]:
```
# Check if multiple values are present in column 'A'
is_present_multiple = df['A'].isin([2, 4, 6])
print("\nAre 2, 4, and 6 present in column 'A' ?: \n", is_present_multiple)
```

```
Are 2, 4, and 6 present in column 'A' ?:
 0    False
1     True
2    False
3     True
4    False
Name: A, dtype: bool
```

In [ ]:
```
# Sample DataFrame
data = {'A': [1, 2, 3, 4, 5],
'B': [5, 4, 3, 2, 1],
'C': [2, 3, 5, 1, 4]}
df = pd.DataFrame(data, index=['a', 'b', 'c', 'd', 'e'])

# Get integer locations for requested labels
index_values = df.index.get_indexer( ['b', 'e', 'f'])
print(index_values) # Output: [ 1 4-1] (-1 indicates 'f' is not in the
index)
```

```
[ 1  4 -1]
```

In [ ]:
```
#histogram values on multiple related columns in a DataFrame

# Sample DataFrame
df = pd.DataFrame({
'coll': [1, 2, 3, 1, 2, 3, 4, 5],
'col2': [2, 3, 4, 2, 3, 4, 5, 6],
'col3': [3, 4, 5, 3, 4, 5, 6, 7]
})

# Calculate histogram values
result = df.apply(pd.value_counts).fillna(0)

# Print the result
print(result)
```

```
   coll  col2  col3
1   2.0   0.0   0.0
2   2.0   2.0   0.0
3   2.0   2.0   2.0
4   1.0   2.0   2.0
5   1.0   1.0   2.0
6   0.0   1.0   1.0
7   0.0   0.0   1.0
```

```
/tmp/ipykernel_2279/3797807260.py:11: FutureWarning: pandas.value_counts is
deprecated and will be removed in a future version. Use
pd.Series(obj).value_counts() instead.
  result = df.apply(pd.value_counts).fillna(0)
```

In [ ]:
```
# 12 - EXPLORATORY DATA ANALYSIS
# Gokul G (241801068)
file =open('/content/adult.data')
first_line = file. readline()

# Process the first line (e.g., print it)
print("First line:\n", first_line)
```

```
First line:
 39, State-gov, 77516, Bachelors, 13, Never-married, Adm-clerical, Not-in-
family, White, Male, 2174, 0, 40, United-States, ⩽50K
```

In [ ]:

```
def chr_int(a):
  if a.isdigit():
    return int(a)
  else:
    return 0

data=[]
for line in file:
  data1=line.split(',')
  if len(data1) == 15:

data.append([chr_int(data1[0]),data1[1],chr_int(data1[2]),data1[3],chr_int(d
ata1[4]),data1[5],data1[6],\
    data1[7],data1[8],data1[9],chr_int(data1[10]),chr_int(data1[11]),
chr_int(data1[12]), data1[13],\
    data1[14]])

print(data[:1])
```

```
[[38, ' Private', 0, ' HS-grad', 0, ' Divorced', ' Handlers-cleaners', '
Not-in-family', ' White', ' Male', 0, 0, 0, ' United-States', ' ≤50K\n']]
```

In [ ]:

```
%matplotlib inline
import pandas as pd

df = pd.DataFrame(data) # Two-dimensional size-mutable, potentially
heterogeneous tabular data structure with labeled axes

df.columns = ['age', 'type_employer', 'fnlwgt', 'education',
"education_num","marital", "occupation", "relationship", "race","sex",
"capital_gain", "capital_loss", "hr_per_week","country","income"]

df.head()
```

Out[13]: Columns 1–8 of 16

	age	type_employer	fnlwgt	education	education_num	marital	occupation
0	38	Private	0	HS-grad	0	Divorced	Handlers-cleaners
1	53	Private	0	11th	0	Married-civ-spouse	Handlers-cleaners
2	28	Private	0	Bachelors	0	Married-civ-spouse	Prof-specialty
3	37	Private	0	Masters	0	Married-civ-spouse	Exec-managerial
4	49	Private	0	9th	0	Married-spouse-absent	Other-service

Columns 9–15 of 16 · first column repeated

	relationship	race	sex	capital_gain	capital_loss	hr_per_week	country
0	Not-in-family	White	Male	0	0	0	United-States
1	Husband	Black	Male	0	0	0	United-States
2	Wife	Black	Female	0	0	0	Cuba
3	Wife	White	Female	0	0	0	United-States
4	Not-in-family	Black	Female	0	0	0	Jamaica

Columns 16–16 of 16 · first column repeated

	income
0	<=50K\n
1	<=50K\n
2	<=50K\n
3	<=50K\n
4	<=50K\n

In [ ]:
```
df.tail()
```

Out[14]: Columns 1–8 of 16

	age	type_employer	fnlwgt	education	education_num	marital	occupation
32554	27	Private	0	Assoc-acdm	0	Married-civ-spouse	Tech-support
32555	40	Private	0	HS-grad	0	Married-civ-spouse	Machine-op-inspct
32556	58	Private	0	HS-grad	0	Widowed	Adm-clerical
32557	22	Private	0	HS-grad	0	Never-married	Adm-clerical
32558	52	Self-emp-inc	0	HS-grad	0	Married-civ-spouse	Exec-managerial

Columns 9–15 of 16 · first column repeated

	relationship	race	sex	capital_gain	capital_loss	hr_per_week	country
32554	Wife	White	Female	0	0	0	United-States
32555	Husband	White	Male	0	0	0	United-States
32556	Unmarried	White	Female	0	0	0	United-States
32557	Own-child	White	Male	0	0	0	United-States
32558	Wife	White	Female	0	0	0	United-States

Columns 16–16 of 16 · first column repeated

	income
32554	<=50K\n
32555	>50K\n
32556	<=50K\n
32557	<=50K\n
32558	>50K\n

In [ ]:
```
df.shape #32561 rows, 15 colums
```

Out[16]:
```
(32559, 15)
```

In [ ]:

```
#1st method
country_counts = df['country'].value_counts()
print(country_counts)
```

```
country
United-States                 29168
Mexico                          643
?                               583
Philippines                     198
Germany                         137
Canada                          121
Puerto-Rico                     114
El-Salvador                     106
India                           100
Cuba                             95
England                          90
Jamaica                          81
South                            80
China                            75
Italy                            73
Dominican-Republic               70
Vietnam                          67
Guatemala                        64
Japan                            62
Poland                           60
Columbia                         59
Taiwan                           51
Haiti                            44
Iran                             43
Portugal                         37
Nicaragua                        34
Peru                             31
France                           29
Greece                           29
Ecuador                          28
Ireland                          24
Hong                             20
Trinadad&Tobago                  19
Cambodia                         19
Thailand                         18
Laos                             18
Yugoslavia                       16
Outlying-US(Guam-USVI-etc)       14
Honduras                         13
Hungary                          13
Scotland                         12
Holand-Netherlands                1
Name: count, dtype: int64
```

In [ ]:

```
#2nd method
counts = df.groupby('country').size()
print(counts)
# also: df.outcome.value_counts()
```

```
country
?                              583
Cambodia                        19
Canada                         121
China                           75
Columbia                        59
Cuba                            95
Dominican-Republic              70
Ecuador                         28
El-Salvador                    106
England                         90
France                          29
Germany                        137
Greece                          29
Guatemala                       64
Haiti                           44
Holand-Netherlands               1
Honduras                        13
Hong                            20
Hungary                         13
India                          100
Iran                            43
Ireland                         24
Italy                           73
Jamaica                         81
Japan                           62
Laos                            18
Mexico                         643
Nicaragua                       34
Outlying-US(Guam-USVI-etc)      14
Peru                            31
Philippines                    198
Poland                          60
Portugal                        37
Puerto-Rico                    114
Scotland                        12
South                           80
Taiwan                          51
Thailand                        18
Trinadad&Tobago                 19
United-States                29168
Vietnam                         67
Yugoslavia                      16
dtype: int64
```

In [ ]:

```
counts = df.groupby('age').size() # grouping by age
print(counts)
```

```
age
17    395
18    550
19    712
20    753
21    720
      ...
85      3
86      1
87      1
88      3
90     43
Length: 73, dtype: int64
```

In [ ]:
```
ml= df[(df.sex == ' Male')] # grouping by male
ml.head()
```

Out[24]: Columns 1–8 of 16

	age	type_employer	fnlwgt	education	education_num	marital	occupation
0	38	Private	0	HS-grad	0	Divorced	Handlers-cleaners
1	53	Private	0	11th	0	Married-civ-spouse	Handlers-cleaners
5	52	Self-emp-not-inc	0	HS-grad	0	Married-civ-spouse	Exec-managerial
7	42	Private	0	Bachelors	0	Married-civ-spouse	Exec-managerial
8	37	Private	0	Some-college	0	Married-civ-spouse	Exec-managerial

Columns 9–15 of 16 · first column repeated

	relationship	race	sex	capital_gain	capital_loss	hr_per_week	country
0	Not-in-family	White	Male	0	0	0	United-States
1	Husband	Black	Male	0	0	0	United-States
5	Husband	White	Male	0	0	0	United-States
7	Husband	White	Male	0	0	0	United-States
8	Husband	Black	Male	0	0	0	United-States

Columns 16–16 of 16 · first column repeated

	income
0	<=50K\n
1	<=50K\n
5	>50K\n
7	>50K\n
8	>50K\n

In [ ]:
```
ml.shape #total number of male
```

Out[25]:
```
(21788, 15)
```

In [ ]:
```
ml1=df[(df.sex == ' Male')&(df.income == ' >50K\n')]
ml1.head()
```

Out[27]:

Columns 1–8 of 16

	age	type_employer	fnlwgt	education	education_num	marital	occupation
5	52	Self-emp-not-inc	0	HS-grad	0	Married-civ-spouse	Exec-managerial
7	42	Private	0	Bachelors	0	Married-civ-spouse	Exec-managerial
8	37	Private	0	Some-college	0	Married-civ-spouse	Exec-managerial
9	30	State-gov	0	Bachelors	0	Married-civ-spouse	Prof-specialty
12	40	Private	0	Assoc-voc	0	Married-civ-spouse	Craft-repair

Columns 9–15 of 16 · first column repeated

	relationship	race	sex	capital_gain	capital_loss	hr_per_week	country
5	Husband	White	Male	0	0	0	United-States
7	Husband	White	Male	0	0	0	United-States
8	Husband	Black	Male	0	0	0	United-States
9	Husband	Asian-Pac-Islander	Male	0	0	0	India
12	Husband	Asian-Pac-Islander	Male	0	0	0	?

Columns 16–16 of 16 · first column repeated

	income
5	>50K\n
7	>50K\n
8	>50K\n
9	>50K\n
12	>50K\n

In [ ]:
```
ml1. shape
```

Out[28]:
```
(6662, 15)
```

In [ ]:
```
fm =df[(df.sex == ' Female')]
fm.head()
```

Out[30]: Columns 1–8 of 16

	age	type_employer	fnlwgt	education	education_num	marital	occupation
2	28	Private	0	Bachelors	0	Married-civ-spouse	Prof-specialty
3	37	Private	0	Masters	0	Married-civ-spouse	Exec-managerial
4	49	Private	0	9th	0	Married-spouse-absent	Other-service
6	31	Private	0	Masters	0	Never-married	Prof-specialty
10	23	Private	0	Bachelors	0	Never-married	Adm-clerical

Columns 9–15 of 16 · first column repeated

	relationship	race	sex	capital_gain	capital_loss	hr_per_week	country
2	Wife	Black	Female	0	0	0	Cuba
3	Wife	White	Female	0	0	0	United-States
4	Not-in-family	Black	Female	0	0	0	Jamaica
6	Not-in-family	White	Female	0	0	0	United-States
10	Own-child	White	Female	0	0	0	United-States

Columns 16–16 of 16 · first column repeated

	income
2	<=50K\n
3	<=50K\n
4	<=50K\n
6	>50K\n
10	<=50K\n

In [ ]:
```
fm. shape
```

Out[32]: (10771, 15)

In [ ]:
```
fm1=df[(df.sex == ' Female')&(df.income == ' >50K\n')]
fm1.head()
```

Out[33]: Columns 1–8 of 16

	age	type_employer	fnlwgt	education	education_num	marital	occupation
6	31	Private	0	Masters	0	Never-married	Prof-specialty
17	43	Self-emp-not-inc	0	Masters	0	Divorced	Exec-managerial
50	47	Private	0	Prof-school	0	Married-civ-spouse	Prof-specialty
65	53	Private	0	HS-grad	0	Married-civ-spouse	Adm-clerical
82	44	Private	0	HS-grad	0	Divorced	Craft-repair

Columns 9–15 of 16 · first column repeated

	relationship	race	sex	capital_gain	capital_loss	hr_per_week	country
6	Not-in-family	White	Female	0	0	0	United-States
17	Unmarried	White	Female	0	0	0	United-States
50	Wife	White	Female	0	0	0	Honduras
65	Wife	White	Female	0	0	0	United-States
82	Not-in-family	White	Female	0	0	0	United-States

Columns 16–16 of 16 · first column repeated

	income
6	>50K\n
17	>50K\n
50	>50K\n
65	>50K\n
82	>50K\n

In [ ]:
```
fm1. shape
```

Out[34]:
```
(1179, 15)
```

In [ ]:
```
df1=df[(df.income == ' >50K\n')]
df1.head()
```

Out[35]:

Columns 1–8 of 16

	age	type_employer	fnlwgt	education	education_num	marital	occupation
5	52	Self-emp-not-inc	0	HS-grad	0	Married-civ-spouse	Exec-managerial
6	31	Private	0	Masters	0	Never-married	Prof-specialty
7	42	Private	0	Bachelors	0	Married-civ-spouse	Exec-managerial
8	37	Private	0	Some-college	0	Married-civ-spouse	Exec-managerial
9	30	State-gov	0	Bachelors	0	Married-civ-spouse	Prof-specialty

Columns 9–15 of 16 · first column repeated

	relationship	race	sex	capital_gain	capital_loss	hr_per_week	country
5	Husband	White	Male	0	0	0	United-States
6	Not-in-family	White	Female	0	0	0	United-States
7	Husband	White	Male	0	0	0	United-States
8	Husband	Black	Male	0	0	0	United-States
9	Husband	Asian-Pac-Islander	Male	0	0	0	India

Columns 16–16 of 16 · first column repeated

	income
5	>50K\n
6	>50K\n
7	>50K\n
8	>50K\n
9	>50K\n

In [ ]:
```
print(df1.shape) #>50K
print('The rate of people with high income is: ',
int(len(df1)/float(len(df))*100), '%.')
print('The rate of men with high income is: ', int(len(ml1)/float(len(ml)
)*100), '%.' )
print('The rate of women with high income is: ', int(len(fm1)/float(len(fm)
)*100), '%.' )
```

```
(7841, 15)
The rate of people with high income is:  24 %.
The rate of men with high income is:  30 %.
The rate of women with high income is:  10 %.
```

In [ ]:
```
print('The average age of men is: ', ml['age'].mean(), '.')
print('The average age of women is: ', fm['age'].mean(), '.')
```

```
The average age of men is:  39.4330824306958 .
The average age of women is:  36.85823043357163 .
```

In [ ]:
```
print('The average age of high-income men is: ', ml1['age'].mean(), '.' )
#>50K
print('The average age of high-income women is: ', fm1['age'].mean(), '.')
#>50K
```

```
The average age of high-income men is:  44.62578805163614 .
The average age of high-income women is:  42.125530110262936 .
```

In [ ]:
```
ml_mu =ml['age'].mean()
ml_mu
```

Out[40]:
```
np.float64(39.4330824306958)
```

In [ ]:
```
ml_mu = ml['age'].mean()
fm_mu = fm['age'].mean()
ml_var = ml['age'].var()
fm_var = fm['age'].var()
ml_std = ml['age'].std()
fm_std =fm['age'].std()
print('Statistics of age for men: mu:', ml_mu, 'var:', ml_var, 'std:',
ml_std)
print('Statistics of age for women: mu:', fm_mu, 'var:', fm_var, 'std:',
fm_std)
```

```
Statistics of age for men: mu: 39.4330824306958 var: 178.7850293378591 std:
13.37105191590621
Statistics of age for women: mu: 36.85823043357163 var: 196.3837063948037
std: 14.01369709943824
```

In [ ]:
```
ml_median= ml['age'].median()
fm_median= fm['age'].median()
print("Median age per men and women: ", ml_median, fm_median)

ml_median_age= ml1['age'].median()
fm_median_age= fm1['age'].median()
print("Median age per men and women with high-income: ", ml_median_age,
fm_median_age) #>50K
```

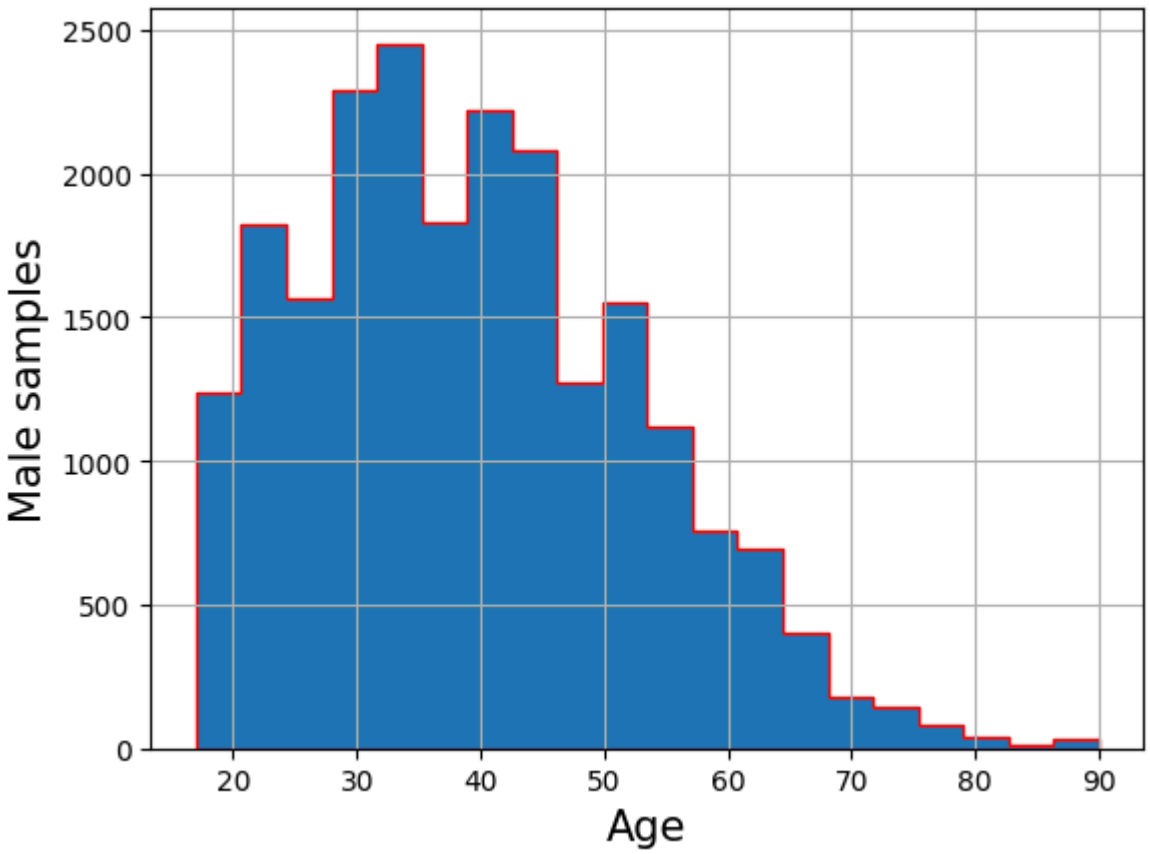
```
Median age per men and women:  38.0 35.0
Median age per men and women with high-income:  44.0 41.0
```

In [ ]:
```
ml_median_hr= ml['hr_per_week'].median()
fm_median_hr= fm['hr_per_week'].median()
print("Median hours per week per men and women: ", ml_median_hr,
fm_median_hr)
```

```
Median hours per week per men and women:  0.0 0.0
```

In [ ]:
```
import matplotlib.pyplot as plt
ml_age=ml['age']
ml_age.hist(density=False, histtype='stepfilled', edgecolor="red", bins=20)

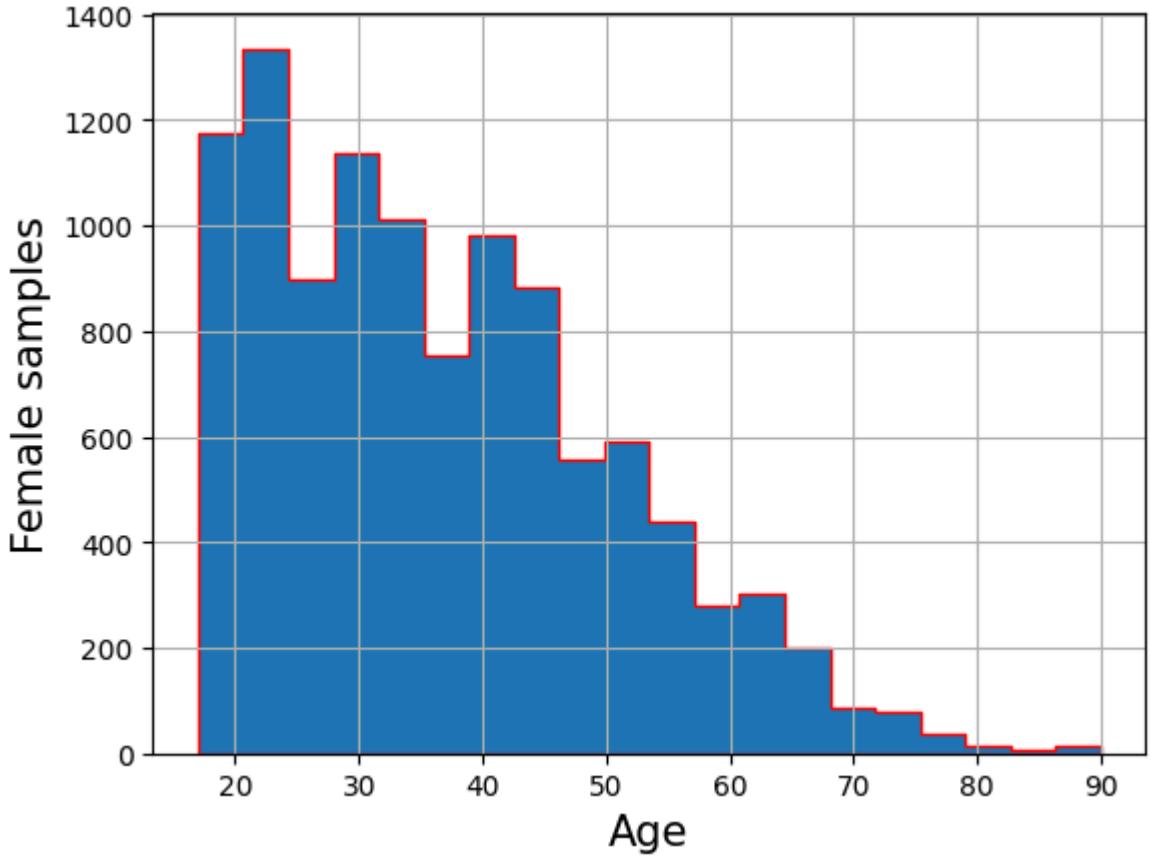
plt.xlabel('Age',fontsize=15)
plt.ylabel('Male samples',fontsize=15)
plt.show()
```



In [ ]:

```
fm_age=fm['age']

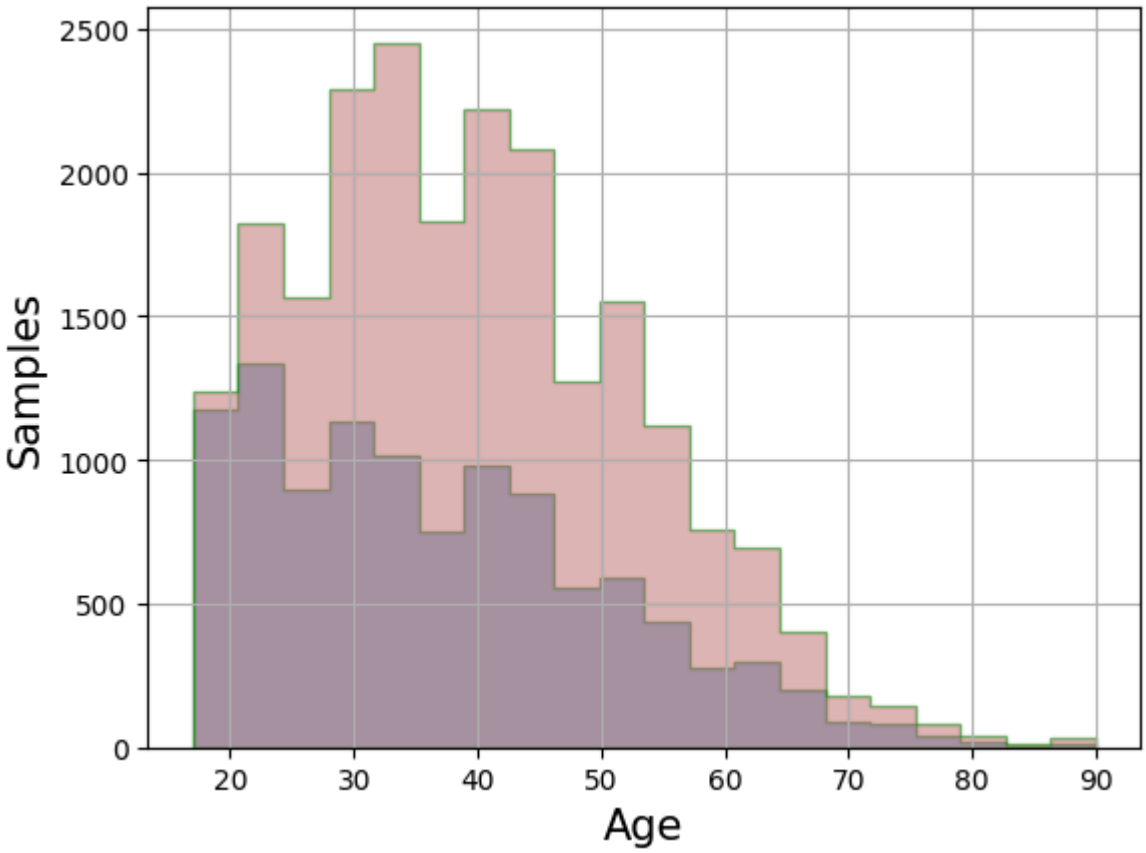
fm_age.hist(density=False, histtype='stepfilled', edgecolor="red", bins=20)
plt.xlabel('Age',fontsize=15)
plt.ylabel('Female samples', fontsize=15)
plt.show()
```



In [ ]:

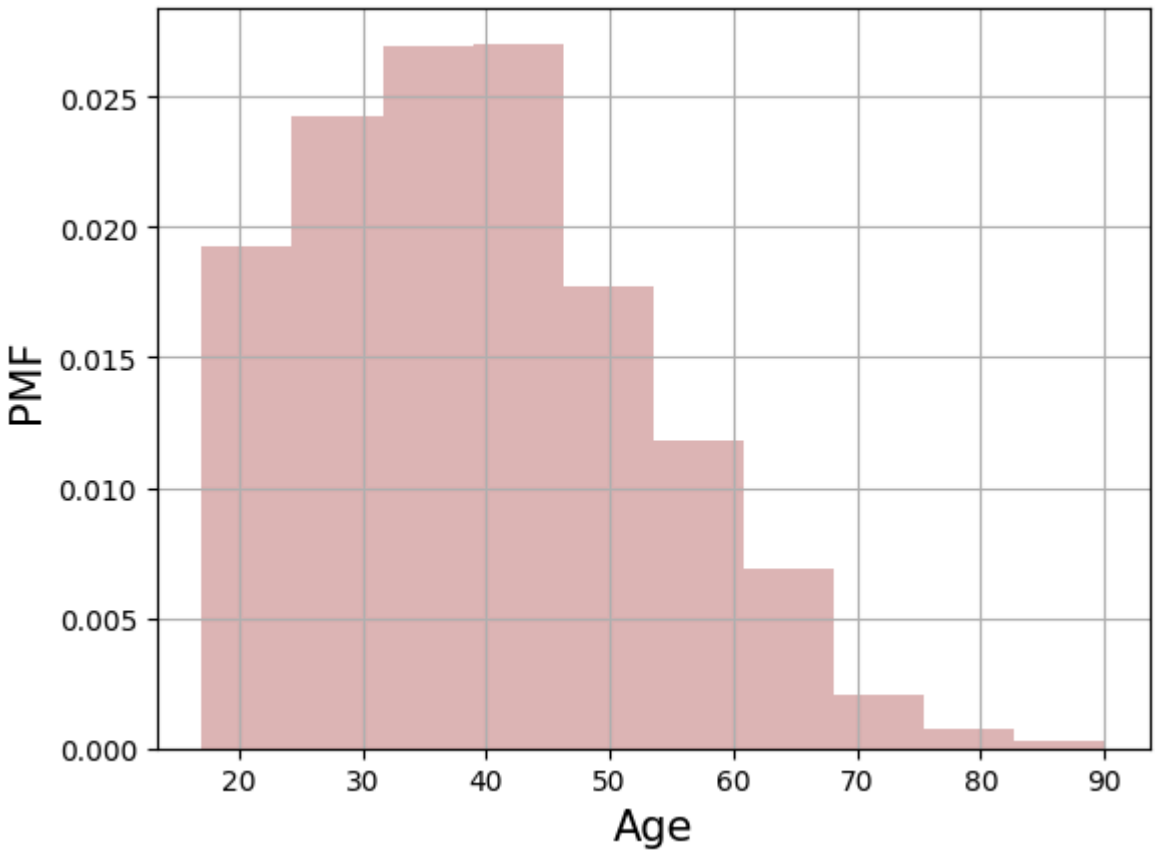
```
import seaborn as sns
import matplotlib.pyplot as plt

fm_age.hist(density=False, histtype='stepfilled', edgecolor="green",
alpha=0.5, bins=20) # default number of bins = 10
ml_age.hist(density=False, histtype='stepfilled', edgecolor="green",
alpha=0.5, color=sns.desaturate("indianred", 0.75), bins=20)
plt.xlabel('Age', fontsize=15)
plt.ylabel('Samples', fontsize=15)
plt.show()
```

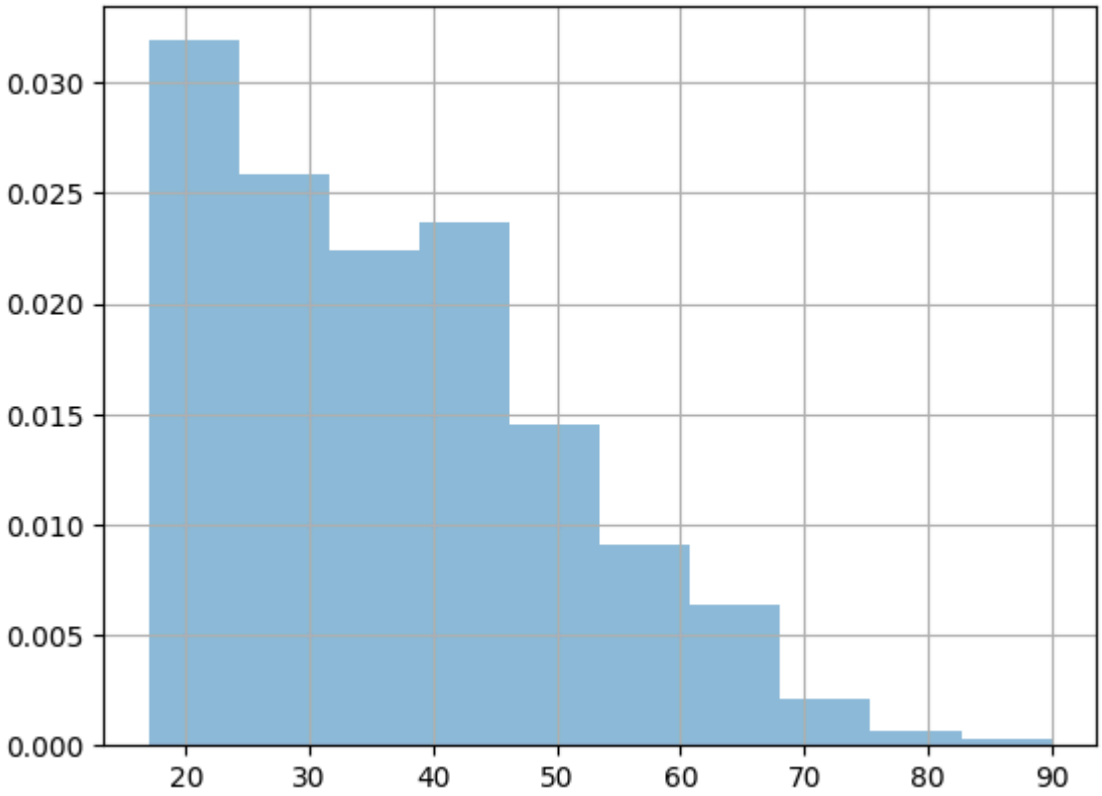


In [ ]:
```
# default number of bins = 10
ml_age.hist(density=True, histtype='stepfilled', alpha =0.5,
color=sns.desaturate("indianred", 0.75), bins=10)
plt.xlabel('Age',fontsize=15)
plt.ylabel('PMF',fontsize=15)
plt.show()

fm_age.hist(density=True, histtype='stepfilled', alpha =0.5, bins=10)
```

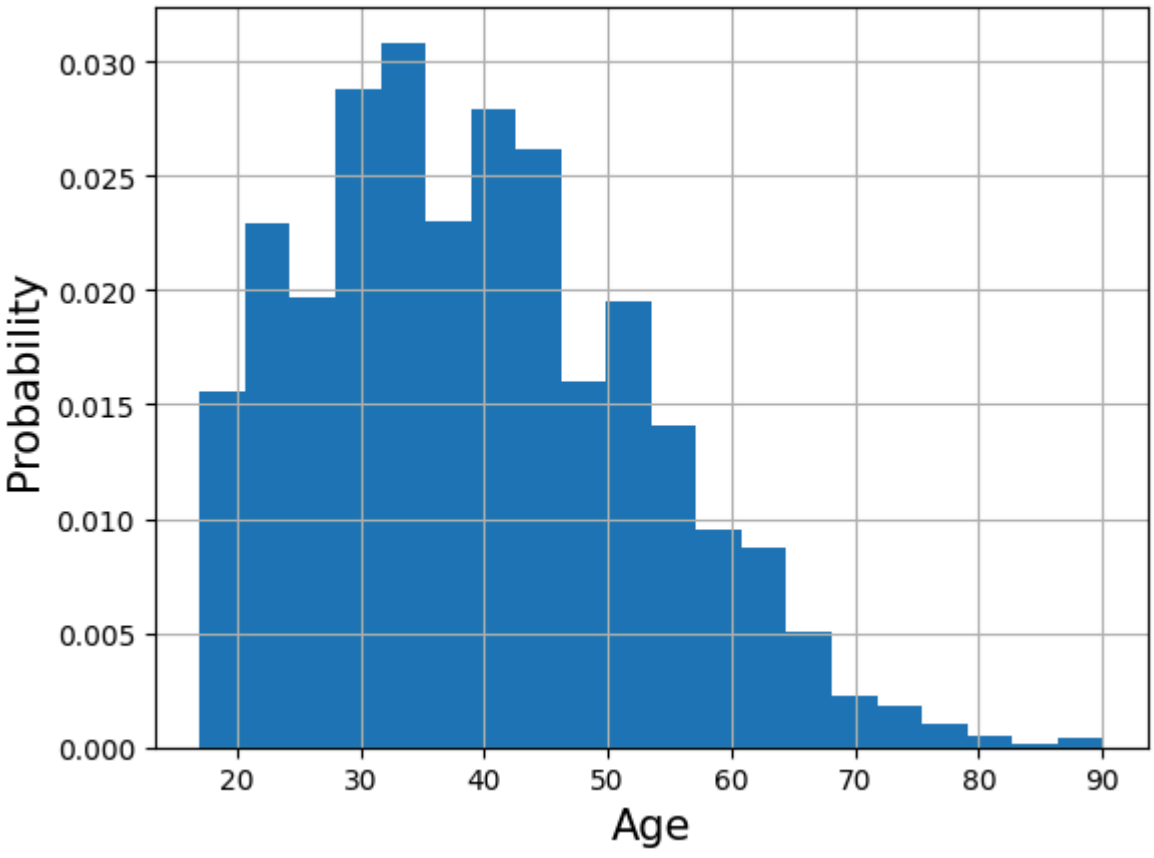


Out[50]:
```
<Axes: >
```



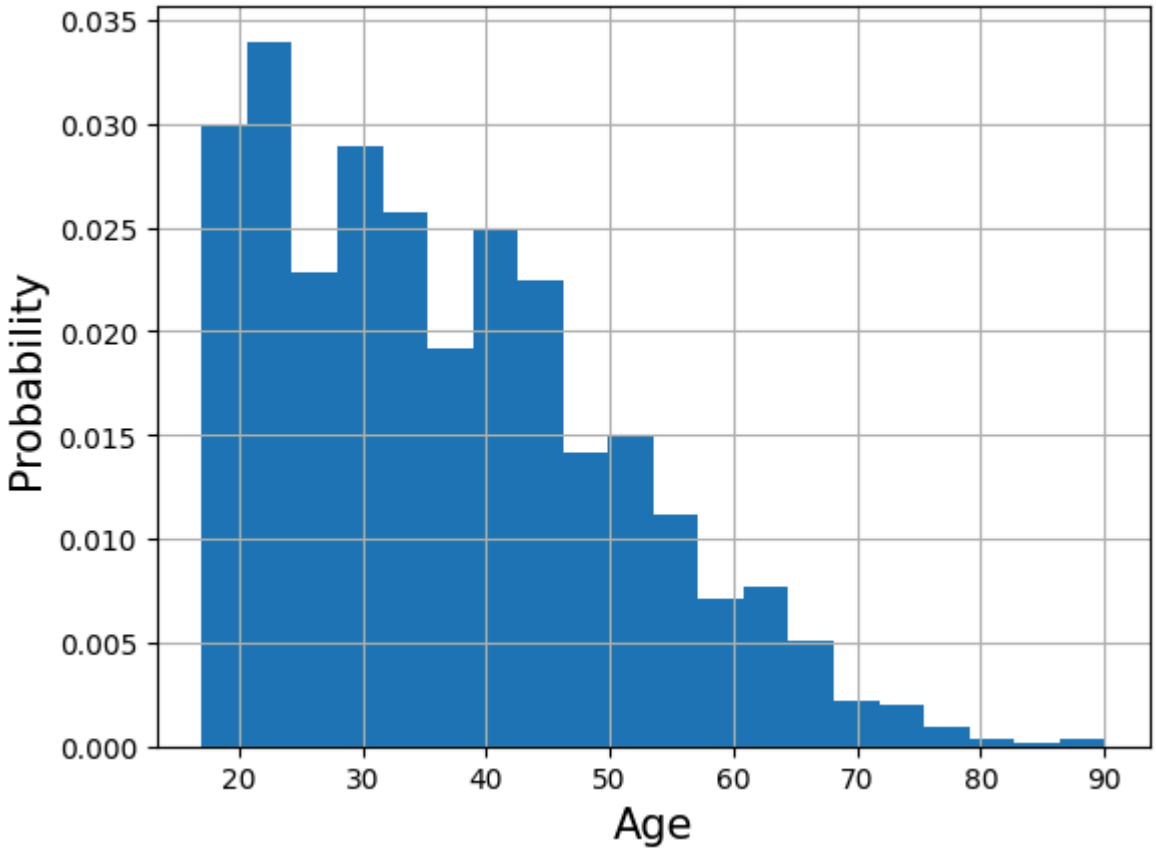
In [ ]:

```
import scipy.stats as stats
ml_age.hist(density=True, histtype='stepfilled', bins=20)
plt.xlabel('Age',fontsize=15)
plt.ylabel('Probability', fontsize=15)
plt.show()
```



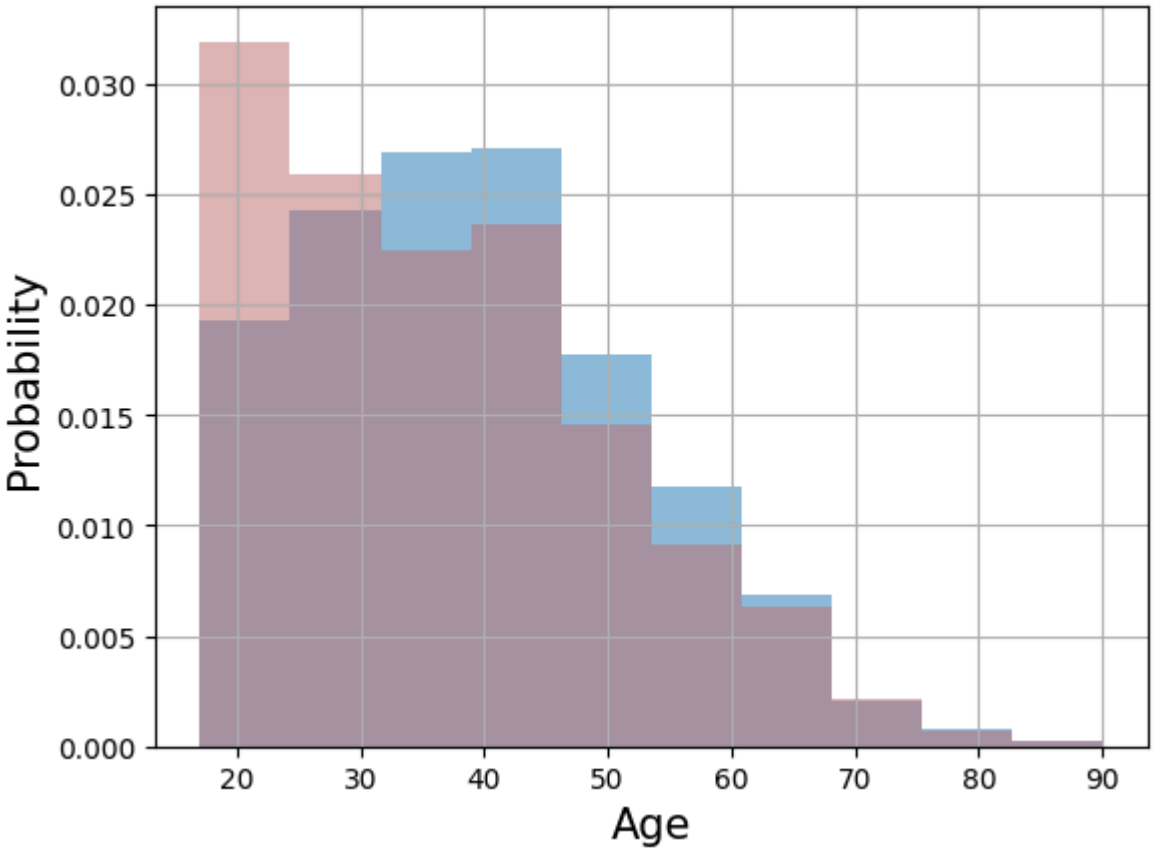
In [ ]:

```
fm_age.hist(density=True, histtype='stepfilled', bins=20)
plt.xlabel('Age',fontsize=15)
plt.ylabel('Probability', fontsize=15)
plt.show()
```



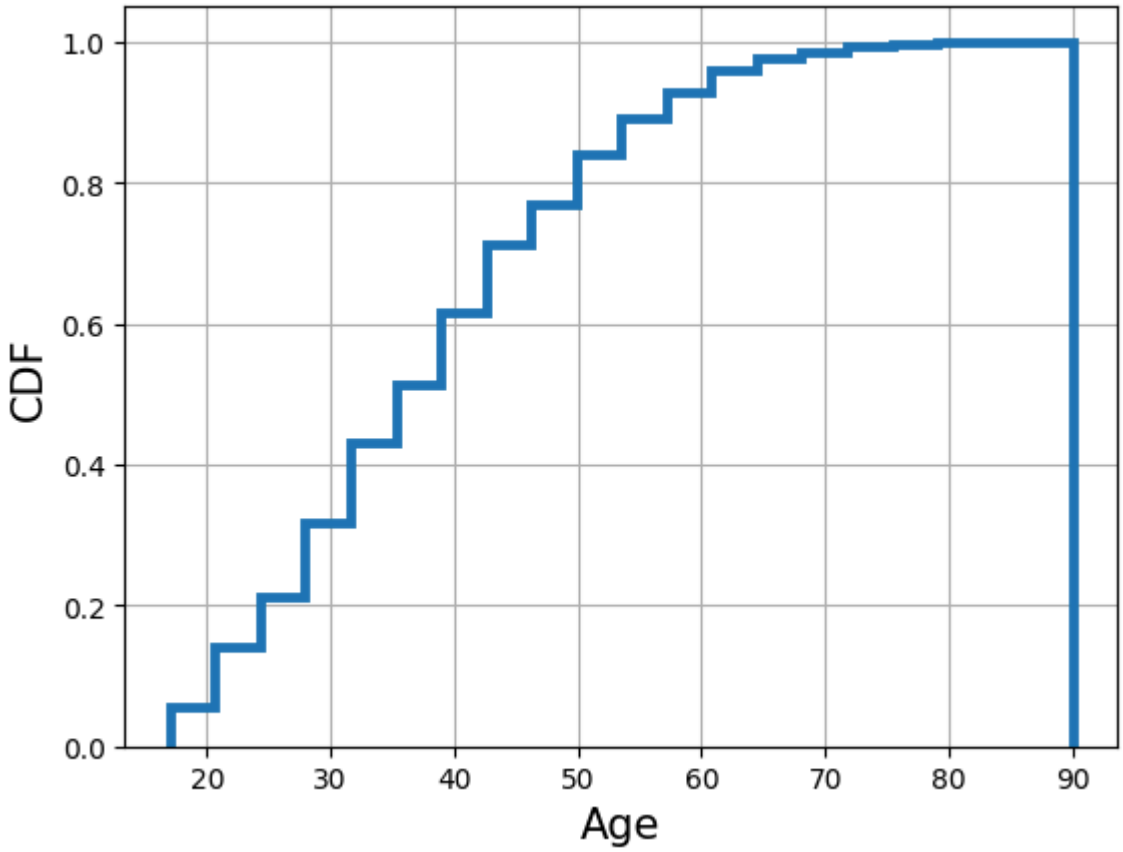
In [ ]:

```
ml_age.hist(bins=10, density=True, histtype='stepfilled', alpha =0.5) #
default number of bins = 10
fm_age.hist(bins=10, density=True, histtype='stepfilled', alpha =0.5,
color=sns.desaturate("indianred", 0.75))
plt.xlabel('Age',fontsize=15)
plt.ylabel('Probability', fontsize=15)
plt.show()
```



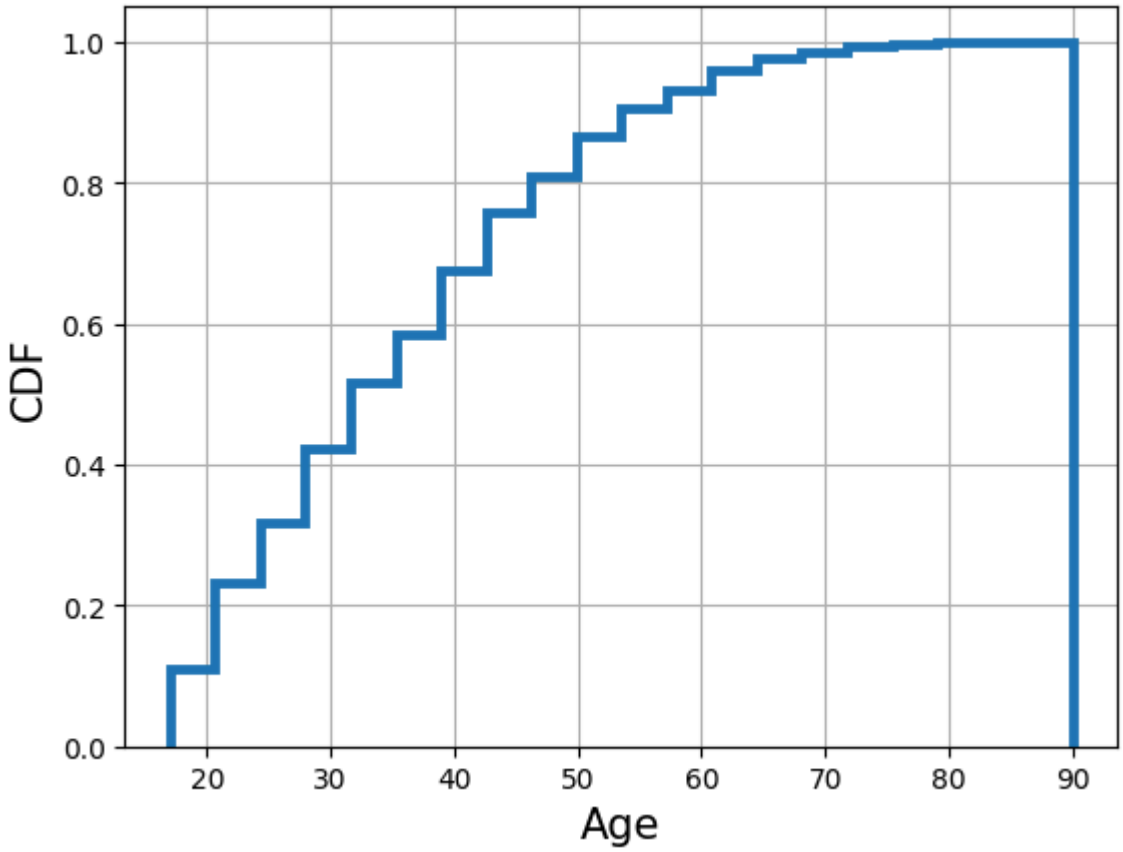
In [ ]:

```
ml_age.hist(density=True, histtype='step', cumulative=True, linewidth=3.5,
bins=20)
plt.xlabel('Age',fontsize=15)
plt.ylabel('CDF',fontsize=15)
plt.show()
```

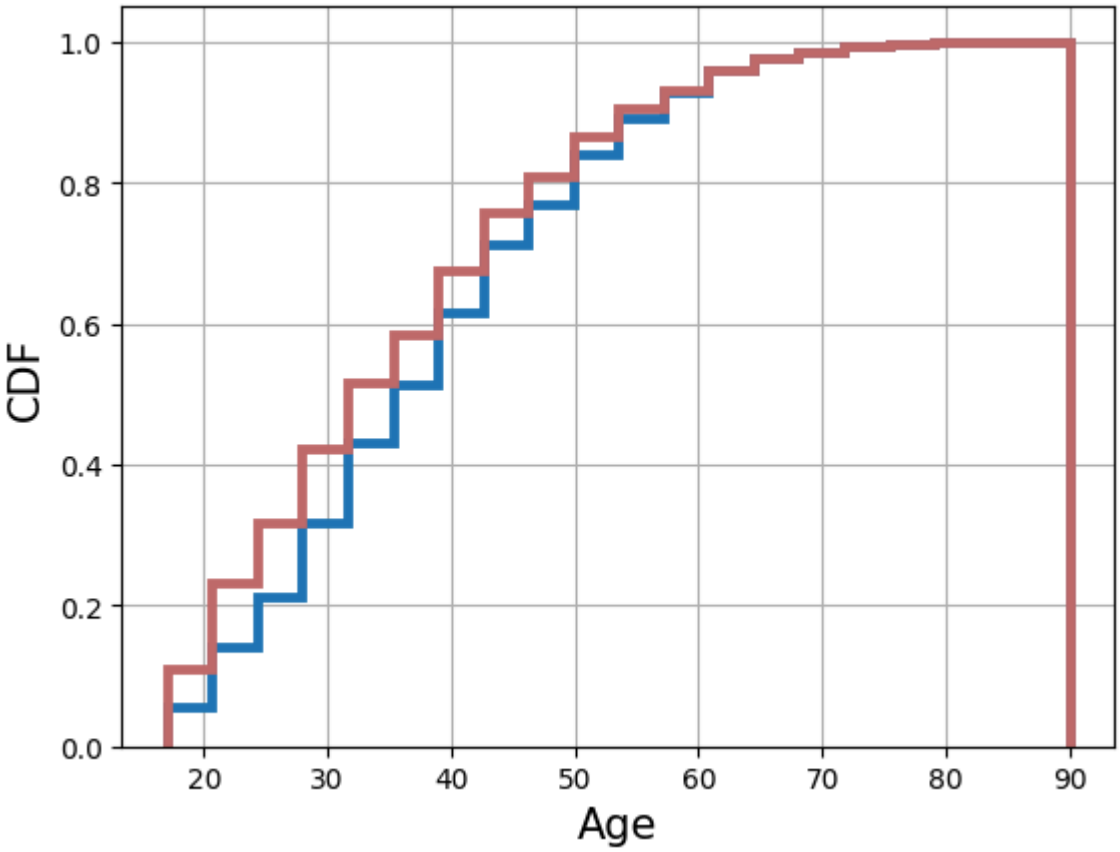


In [ ]:

```
fm_age.hist(density=True, histtype='step', cumulative=True, linewidth=3.5,
bins=20)
plt.xlabel('Age',fontsize=15)
plt.ylabel('CDF',fontsize=15)
plt.show()
```



In [ ]:
```
ml_age.hist(density=True, histtype='step', cumulative=True, linewidth=3.5,
bins=20)
fm_age.hist(density=True, histtype='step', cumulative=True, linewidth=3.5,
bins=20, color=sns.desaturate("indianred", .75))
plt.xlabel('Age',fontsize=15)
plt.ylabel('CDF',fontsize=15)
plt.show()
```



In [ ]:
```
print("The mean sample difference is ", ml_age.mean() - fm_age.mean())
```

```
The mean sample difference is  2.5748519971241706
```

In [ ]:
```
df['age'].median()
```

Out[60]:
```
37.0
```

In [ ]:
```
#income >50k and Age is less than (median - 15)
len(df[(df.income == ' >50K\n') & (df['age'] < df['age'].median() - 15)])
```

Out[62]:
```
5
```

In [ ]:
```
#Income is >50K and Age is above (median + 35)
len(df[(df.income == ' >50K\n') & (df['age'] > df['age'].median() + 35)])
```

Out[63]:
```
69
```

In [ ]:
```
df2 = df.drop(df.index[(df.income == ' >50K\n') &
(df['age']>df['age'].median() +35) & (df['age'] > df['age'].median() -15)])
df2.shape #after dropping outliers
```

Out[66]:
```
(32490, 15)
```

In [ ]:
```
ml1_age=ml1['age'] #>50K income
fm1_age=fm1['age'] #>50K income
#drop the outliers in high income population from age column alone
ml2_age =ml1_age.drop(ml1_age.index[(ml1_age>df['age'].median()+35) &
(ml1_age > df['age'].median() - 15)])
print(ml2_age.shape)
fm2_age = fm1_age.drop(fm1_age.index[(fm1_age>df['age'].median()+35) &
(fm1_age>df['age'].median() - 15)])
print(fm2_age.shape)
```

```
(6601,)
(1171,)
```

In [ ]:
```
mu2ml = ml2_age.mean() #after dropping outliers in high income population
std2ml = ml2_age.std()
md2ml = ml2_age.median()
# Computing the mean, std, median, min and max for the high-income male
population
print("Men statistics: Mean:", mu2ml, "Std:", std2ml, "Median:", md2ml,
"Min:", ml2_age.min(), "Max:", ml2_age.max())
```

```
Men statistics: Mean: 44.317982123920615 Std: 10.019749857171412 Median:
44.0 Min: 19 Max: 72
```

In [ ]:
```
mu2fm = fm2_age.mean() #after dropping outliers in high income population
std2fm = fm2_age.std()
md2fm = fm2_age.median()
# Computing the mean, std, median, min and max for the high-income female
population
print("Women statistics: Mean:", mu2fm, "Std:", std2fm, "Median:", md2fm,
"Min:", fm2_age.min(), "Max:", fm2_age.max())
```

```
Women statistics: Mean: 41.877028181041844 Std: 10.036441807343714 Median:
41.0 Min: 19 Max: 72
```

In [ ]:
```
print(ml_age.mean())
print(fm_age.mean())
```

```
39.4330824306958
36.85823043357163
```

In [ ]:
```
print('The mean difference with outliers is: %4.2f.' % (ml_age.mean() -
fm_age.mean()))
```

```
The mean difference with outliers is: 2.57.
```

In [ ]:
```
print("The mean difference without outliers is: %4.2f." % (ml2_age.mean() -
fm2_age.mean()))
```

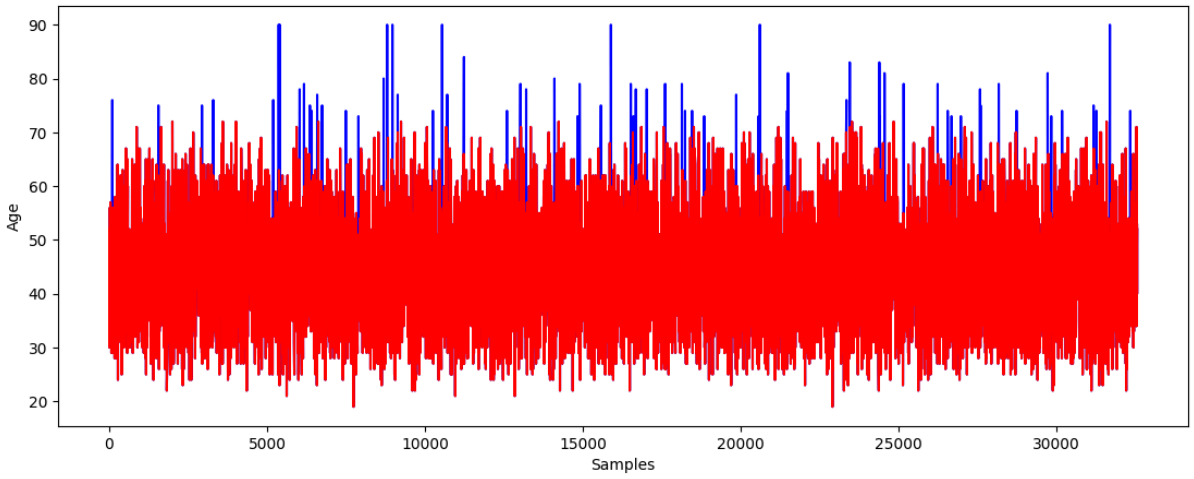
```
The mean difference without outliers is: 2.44.
```

In [ ]:
```
plt.figure(figsize=(13.4,5))

df.age[(df.income == ' >50K\n')].plot(color='blue') #with outlier
df2.age[(df2.income == ' >50K\n')].plot(color='red')# without outlier

plt.ylabel('Age')
plt.xlabel('Samples')
```

Out[76]:
```
Text(0.5, 0, 'Samples')
```



In [ ]:
```
import numpy as np
#higher income population after removing outliers
countx, divisionx = np.histogram(ml2_age, density=True)
county, divisiony = np.histogram(fm2_age, density=True)
print(divisionx)# bin or interval
print(divisiony)
print(county)
```

```
[19.  24.3 29.6 34.9 40.2 45.5 50.8 56.1 61.4 66.7 72. ]
[19.  24.3 29.6 34.9 40.2 45.5 50.8 56.1 61.4 66.7 72. ]
[0.00273915 0.01804618 0.02723039 0.04334305 0.0327087  0.02626364
 0.02030195 0.01111774 0.00467267 0.00225577]
```

In [ ]:
```
countx
```

Out[78]:
```
array([0.00125767, 0.0096612 , 0.02309541, 0.03804455, 0.03395712,
       0.03147036, 0.02669693, 0.01469188, 0.00651702, 0.00328709])
```

In [ ]:
```
val= [(divisionx[i]+divisionx[i+1])/2 for i in range(len(divisionx)-1)]
val
```

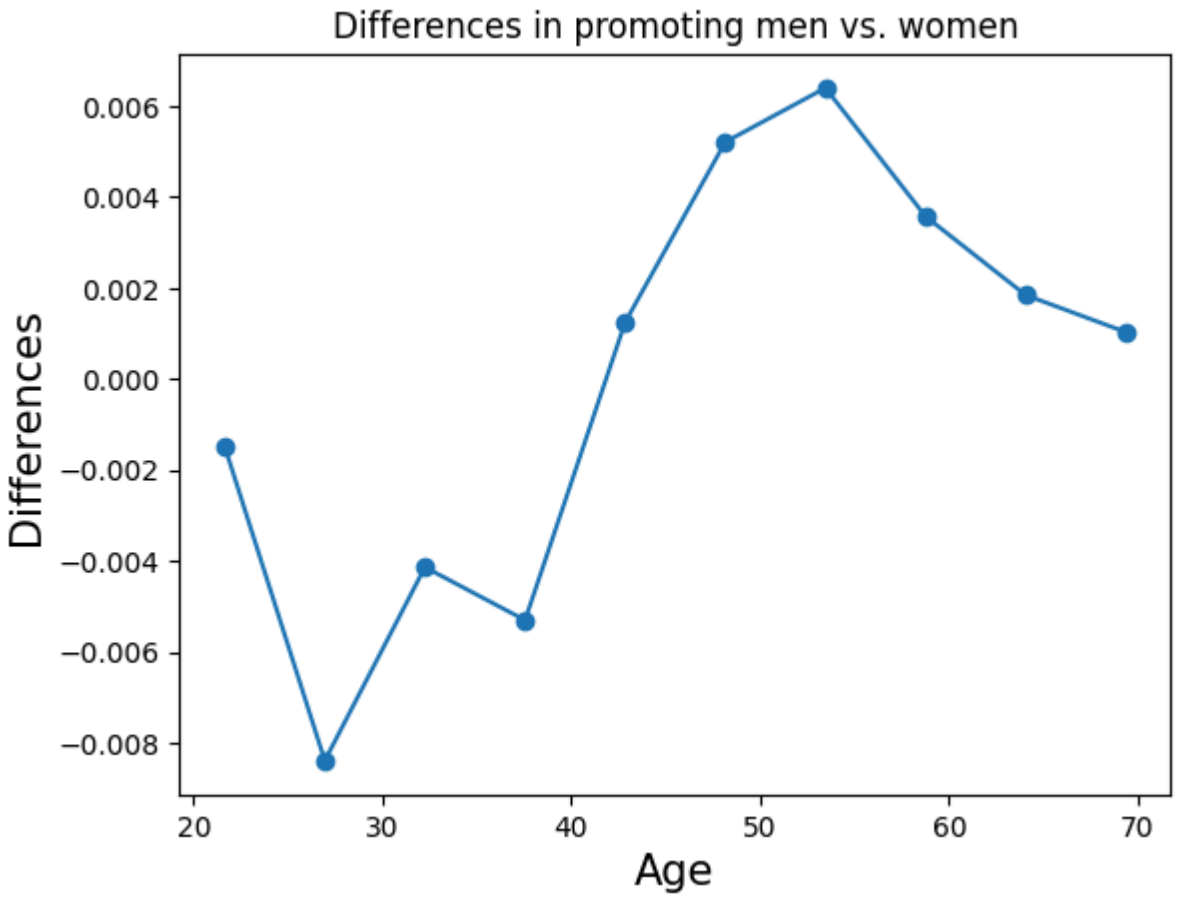
Out[79]:
```
[np.float64(21.65),
 np.float64(26.950000000000003),
 np.float64(32.25),
 np.float64(37.55),
 np.float64(42.85),
 np.float64(48.15),
 np.float64(53.45),
 np.float64(58.75),
 np.float64(64.05),
 np.float64(69.35)]
```

In [ ]:
```
countx-county
```

Out[80]:
```
array([-0.00148148, -0.00838498, -0.00413498, -0.0052985 ,  0.00124842,
        0.00520673,  0.00639498,  0.00357415,  0.00184435,  0.00103132])
```

In [ ]:
```
import matplotlib.pyplot as plt

plt.plot(val, countx-county, 'o-')
plt.title('Differences in promoting men vs. women')
plt.xlabel('Age',fontsize=15)
plt.ylabel('Differences',fontsize=15)
plt.show()
```



In [ ]:
```
# The difference between the mean values of male and female populations.
print("Remember:\n We have the following mean values for men, women and the
difference:\nOriginally: ", ml_age.mean(), fm_age.mean(), ml_age.mean() -
fm_age.mean())
# The difference between the mean values of male and female populations.
print("For high-income: ", ml1_age.mean(), fm1_age.mean(), ml1_age.mean()-
fm1_age.mean())
# The difference between the mean values of male and female populations.
print("After cleaning: ", ml2_age.mean(), fm2_age.mean(), ml2_age.mean()-
fm2_age.mean())

print("\nThe same for the median:")
# The difference between the mean values of male and female populations.
print(ml_age.median(), fm_age.median(), ml_age.median()- fm_age.median())
# The difference between the mean values of male and female populations.
print(ml1_age.median(), fm1_age.median(), ml1_age.median()-
fm1_age.median())
# The difference between the mean values of male and female populations.
print(ml2_age.median(), fm2_age.median(), ml2_age.median()-
fm2_age.median())
```

```
Remember:
 We have the following mean values for men, women and the difference:
Originally:  39.4330824306958 36.85823043357163 2.5748519971241706
For high-income:  44.62578805163614 42.125530110262936 2.5002579413732064
After cleaning:  44.317982123920615 41.877028181041844 2.440953942878771

The same for the median:
38.0 35.0 3.0
44.0 41.0 3.0
44.0 41.0 3.0
```

In [ ]:
```
def skewness(x):
  res=0
  m=x.mean()
  s=x.std()
  for i in x:
    res+=(i-m)*(i-m)*(i-m)
  res/=(len(x)*s*s*s)
  return res

print("The skewness of the male population is:", skewness(ml2_age)) #after
removing outliers
print("The skewness of the female population is:", skewness(fm2_age) )#after
removing outliers
```

```
The skewness of the male population is: 0.2664443838432819
The skewness of the female population is: 0.38633352491285977
```

In [ ]:
```
def pearson(x):
  return 3*(x.mean()-x.median())/x.std()

print("The Pearson's coefficient of the male population is:",
pearson(ml2_age))
print("The Pearson's coefficient of the female population is:",
pearson(fm2_age))
```

```
The Pearson's coefficient of the male population is: 0.0952066054901639
The Pearson's coefficient of the female population is: 0.2621531209596965
```

In [ ]:
```
x=np.arange(0,2.5,0.1)
x
```

Out[89]:
```
array([0. , 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1. , 1.1, 1.2,
       1.3, 1.4, 1.5, 1.6, 1.7, 1.8, 1.9, 2. , 2.1, 2.2, 2.3, 2.4])
```

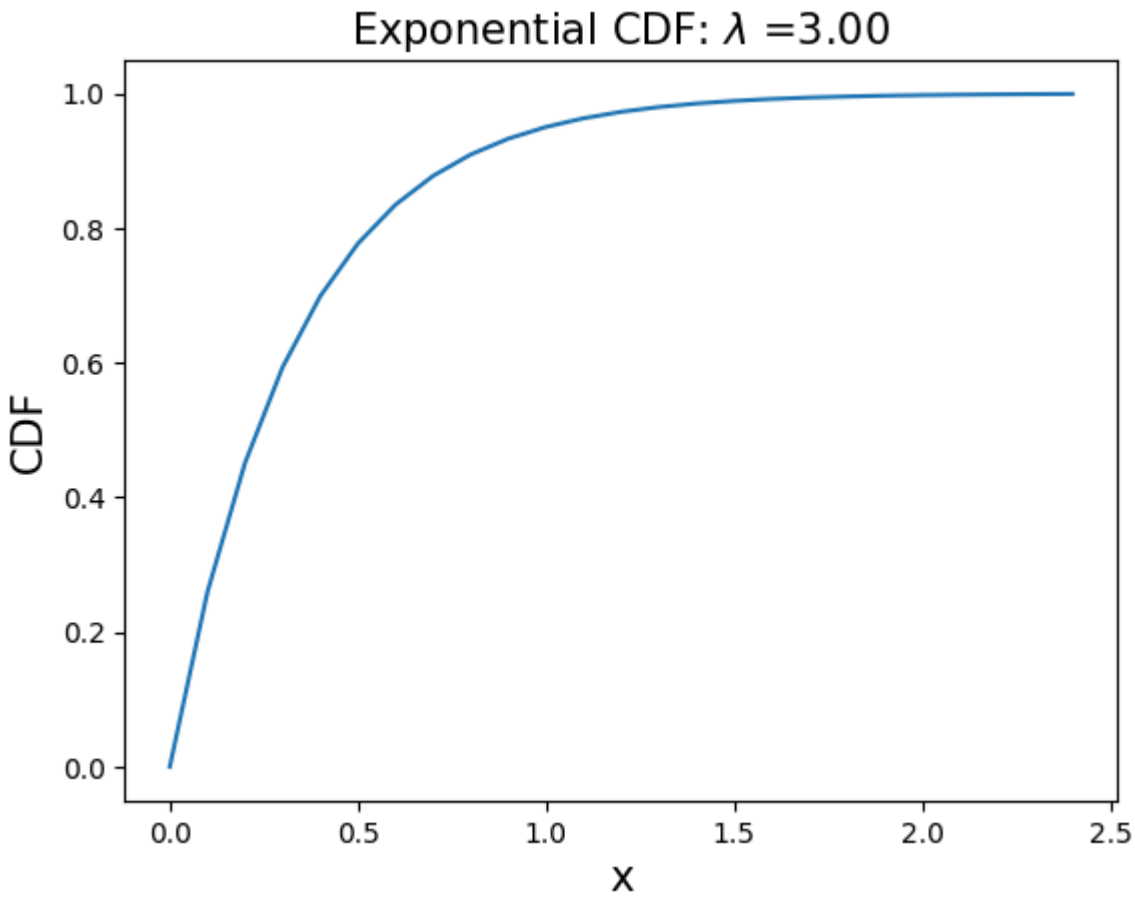
In [ ]:
```
l = 3
y= 1 - np.exp(-l*x)
y
```

Out[91]:
```
array([0.        , 0.25918178, 0.45118836, 0.59343034, 0.69880579,
       0.77686984, 0.83470111, 0.87754357, 0.90928205, 0.93279449,
       0.95021293, 0.96311683, 0.97267628, 0.97975809, 0.98500442,
       0.988891  , 0.99177025, 0.99390325, 0.99548342, 0.99665403,
       0.99752125, 0.9981637 , 0.99863963, 0.99899221, 0.99925341])
```

In [ ]:

```
plt.plot(x,y,'-')
plt.title('Exponential CDF: $\lambda$ =%.2f' % l ,fontsize=15)
plt.xlabel('x',fontsize=15)
plt.ylabel('CDF',fontsize=15) #continous distributions functions (CDF)
plt.show()
```

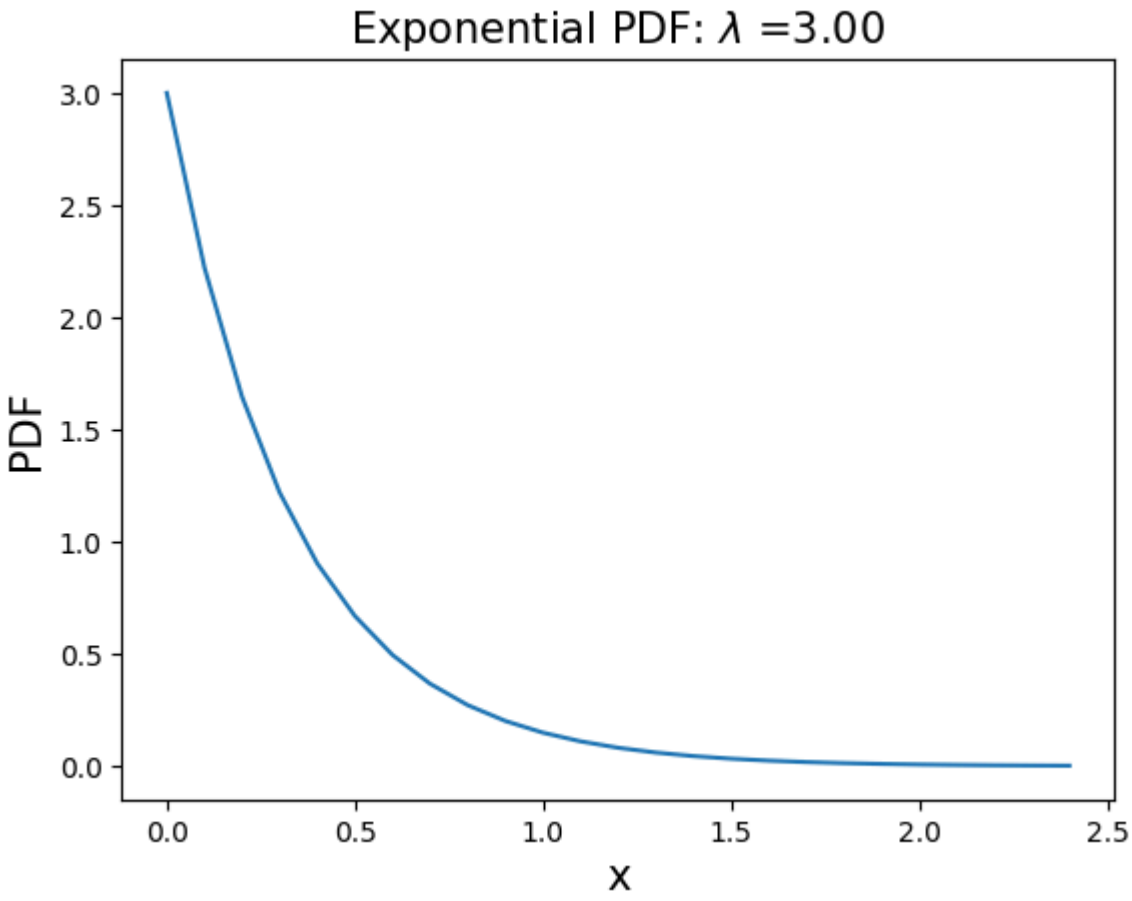
```
<>:2: SyntaxWarning: invalid escape sequence '\l'
<>:2: SyntaxWarning: invalid escape sequence '\l'
/tmp/ipykernel_2666/2630257916.py:2: SyntaxWarning: invalid escape sequence
'\l'
  plt.title('Exponential CDF: $\lambda$ =%.2f' % l ,fontsize=15)
```



In [ ]:

```
from __future__ import division
import scipy. stats as stats
l = 3
x=np.arange(0,2.5,0.1)
y=l* np.exp(-l*x)
plt.plot(x,y,'-')
plt.title('Exponential PDF: $\lambda$ =%.2f' % l, fontsize=15)
plt.xlabel('x', fontsize=15)
plt.ylabel('PDF', fontsize=15) #probability density function
plt. show()
```

```
<>:7: SyntaxWarning: invalid escape sequence '\l'
<>:7: SyntaxWarning: invalid escape sequence '\l'
/tmp/ipykernel_2666/3197336424.py:7: SyntaxWarning: invalid escape sequence
'\l'
  plt.title('Exponential PDF: $\lambda$ =%.2f' % l, fontsize=15)
```

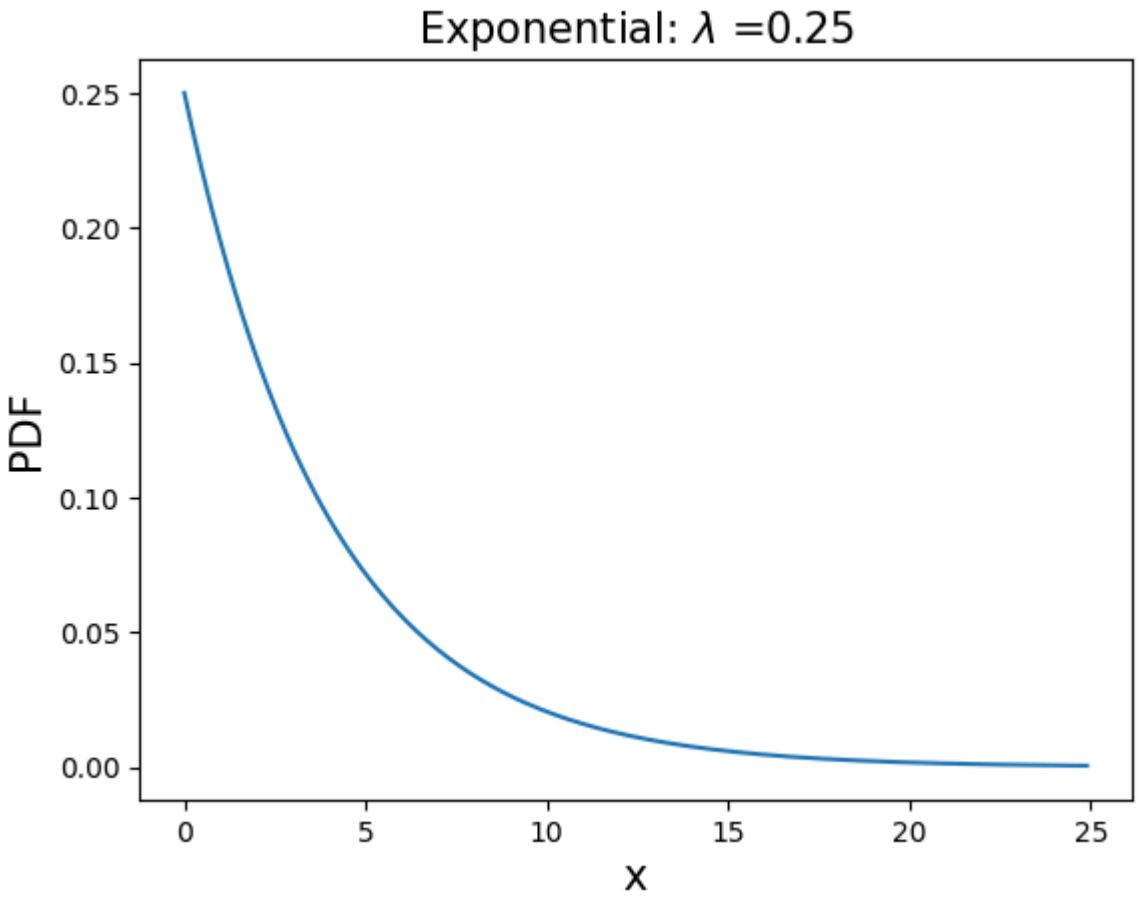


In [ ]:

```
l = 0.25
x=np.arange(0,25,0.1)
y= l * np.exp(-l*x)
plt.plot(x,y,'-')

plt.title('Exponential: $\lambda$ =%.2f' % l , fontsize=15)
plt.xlabel('x',fontsize=15)
plt.ylabel('PDF',fontsize=15)
plt.show()
```

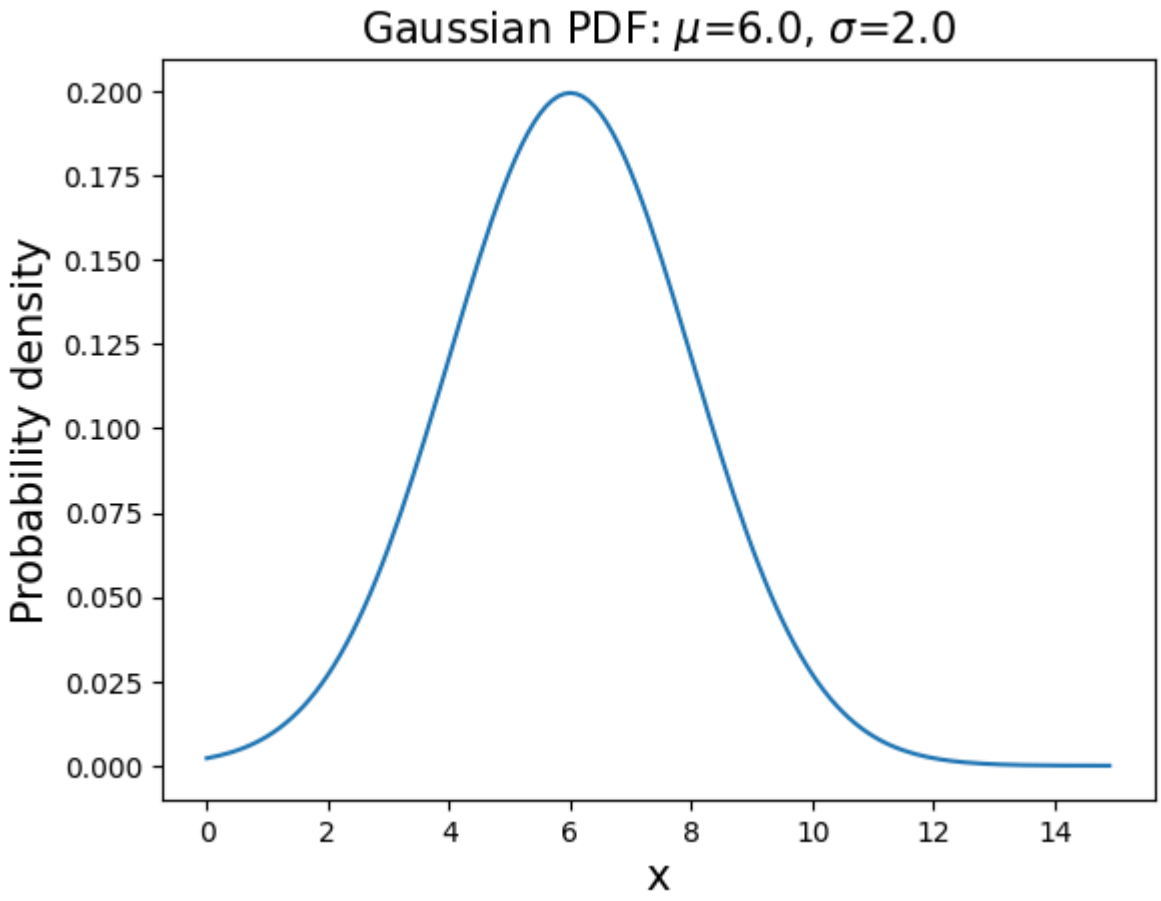
```
<>:6: SyntaxWarning: invalid escape sequence '\l'
<>:6: SyntaxWarning: invalid escape sequence '\l'
/tmp/ipykernel_2666/3130996178.py:6: SyntaxWarning: invalid escape sequence
'\l'
  plt.title('Exponential: $\lambda$ =%.2f' % l , fontsize=15)
```



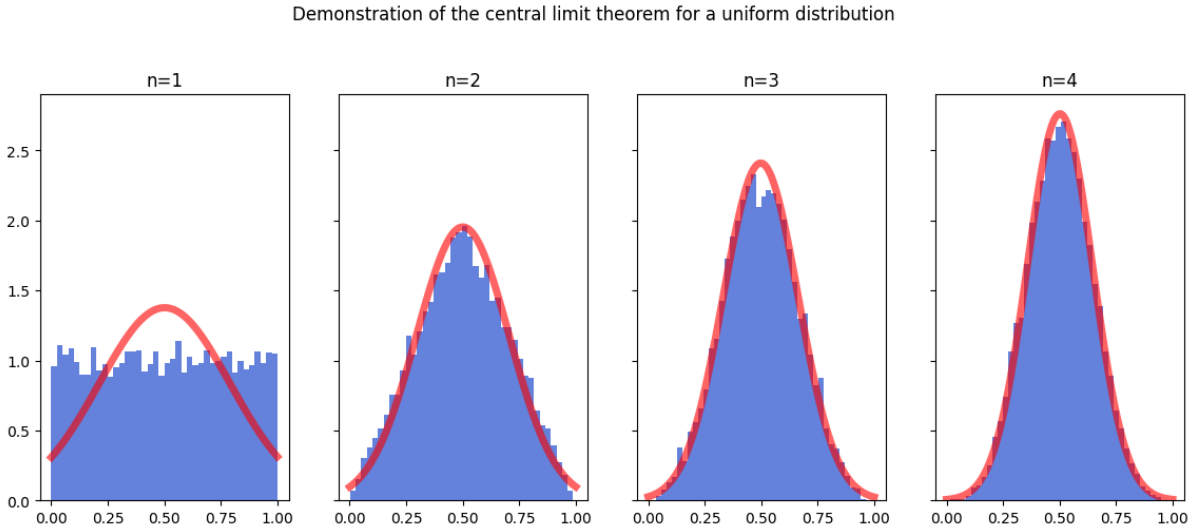
In [ ]:

```
u=6 # mean
s=2 # standard deviation
x=np.arange(0,15,0.1)
y=(1/(np.sqrt(2*np.pi*s*s)))*np.exp(-(((x-u) ** 2)/(2*s*s)))
plt.plot(x,y,'-')
plt.title('Gaussian PDF: $\mu$=%.1f, $\sigma$=%.1f' % (u,s), fontsize=15)
plt.xlabel('x',fontsize=15)
plt.ylabel('Probability density', fontsize=15)
plt.show()
```

```
<>:6: SyntaxWarning: invalid escape sequence '\m'
<>:6: SyntaxWarning: invalid escape sequence '\m'
/tmp/ipykernel_2666/668446035.py:6: SyntaxWarning: invalid escape sequence
'\m'
  plt.title('Gaussian PDF: $\mu$=%.1f, $\sigma$=%.1f' % (u,s), fontsize=15)
```



In [ ]:
```
fig, ax = plt.subplots(1, 4, sharey=True, squeeze=True, figsize=(14, 5))
# 4 sub plots
x = np.linspace(0, 1, 100)
for i in range(4):
  f= np.mean(np.random.random((10000, i+1)), 1)
  m, s = np.mean(f), np.std(f, ddof=1) #mean, standard deviation
  fn =(1/(s*np.sqrt(2*np.pi)))*np.exp(-(x-m) ** 2/(2*s ** 2)) # normal pdf
  ax[i].hist(f, 40, density=True, color=[0, 0.2, .8, .6])
  ax[i].set_title('n=%d' %(i+1))
  ax[i].plot(x, fn, color=[1, 0, 0, .6], linewidth=5)
plt.suptitle('Demonstration of the central limit theorem for a uniform
distribution', y=1.05)
plt.show()
```



In [ ]:
```
from scipy.stats.distributions import norm
x1 = np.random.normal(-1, 2, 15) #normal distribution
# parameters: (loc=0.0 (peak), scale=1.0 (std deviation), size=15)
x2 = np.random.normal(6, 3, 10)
x1
```

Out[103]:
```
array([-2.77321878,  1.05835302,  1.00486633, -1.5643589 , -2.9043645 ,
       -1.39559073, -1.77220272, -1.04849567,  1.70322708, -1.17719494,
       -3.26575878, -0.0084772 ,  0.24580673, -2.28934903, -2.33713356])
```

In [ ]:
```
x2
```

Out[104]:
```
array([6.94461001, 6.72837806, 6.41550079, 3.5031035 , 5.19647166,
       2.48355255, 6.75721023, 2.31347906, 4.66045501, 7.57965142])
```

In [ ]:
```
# r_ Translates slice objects to concatenation along the first axis.
y = np.r_[x1, x2]

#Adjust X Values Based on y
x= np.linspace(min(y), max(y), 100)

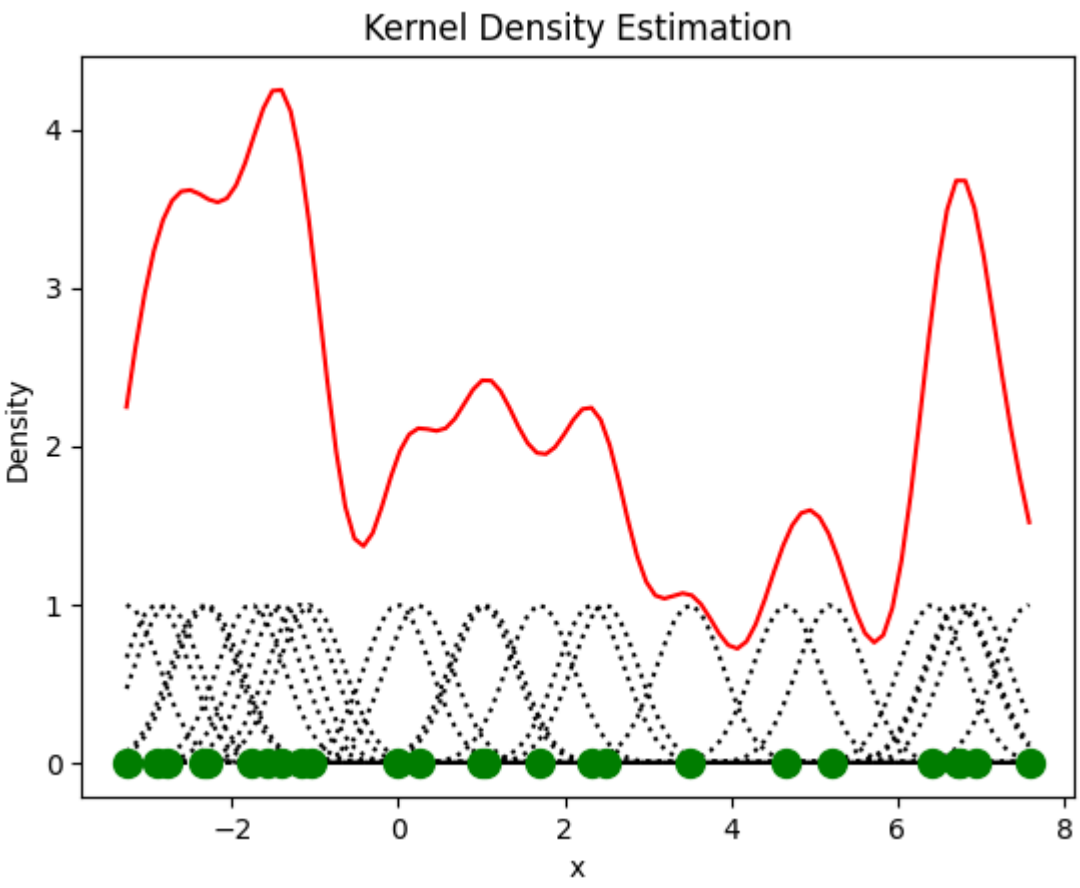
# Smoothing parameter
s = 0.4

# Calculate the kernels
kernels = np.transpose([norm.pdf(x, yi, s) for yi in y]) #normal
distribution, pdf
kernels.shape
```

Out[108]:
```
(100, 25)
```

In [ ]:
```
plt.plot(x, kernels, 'k:') # Individual kernels as black dotted lines
plt.plot(x, kernels.sum(1), 'r') # Sum of kernels as a solid red line
plt.plot(y, np.zeros(len(y)), 'go', ms=10)# Original data points as green
dots
plt.title('Kernel Density Estimation')
plt.xlabel('x')
plt.ylabel('Density')
```

Out[110]:
```
Text(0, 0.5, 'Density')
```



In [ ]:

```
#from scipy.stats import kde

x1 = np.random.normal(-1, 0.5, 15) # parameters: (loc=0.0, scale=1.0,
size=None)
x2 = np.random.normal(6, 1, 10)
y = np.r_[x1, x2] # r_ Translates slice objects to concatenation along the
first axis.
x= np.linspace(min(y), max(y), 100)

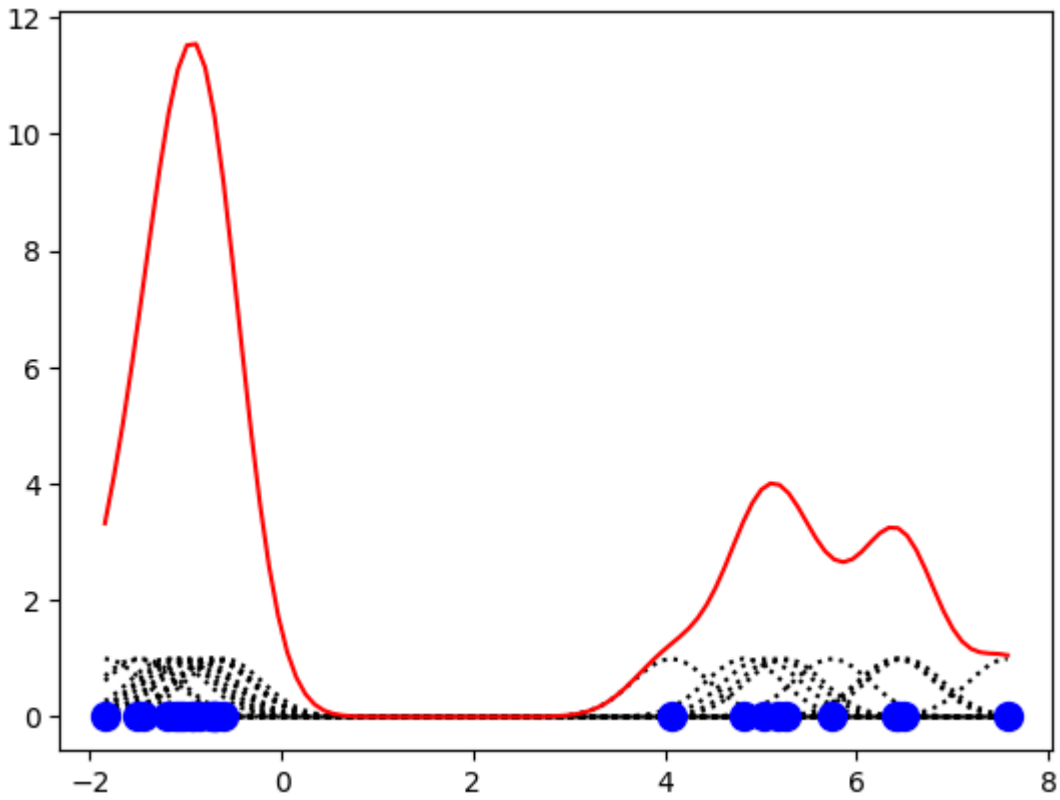
s = 0.4 # Smoothing parameter

kernels = np.transpose( [norm.pdf(x, yi, s) for yi in y]) # Calculate the
kernels
#density = kde.gaussian_kde(y)

plt.plot(x, kernels, 'k:')
plt.plot(x, kernels.sum(1), 'r')
plt.plot(y, np.zeros(len(y)), 'bo', ms=10)
```

Out[111]:

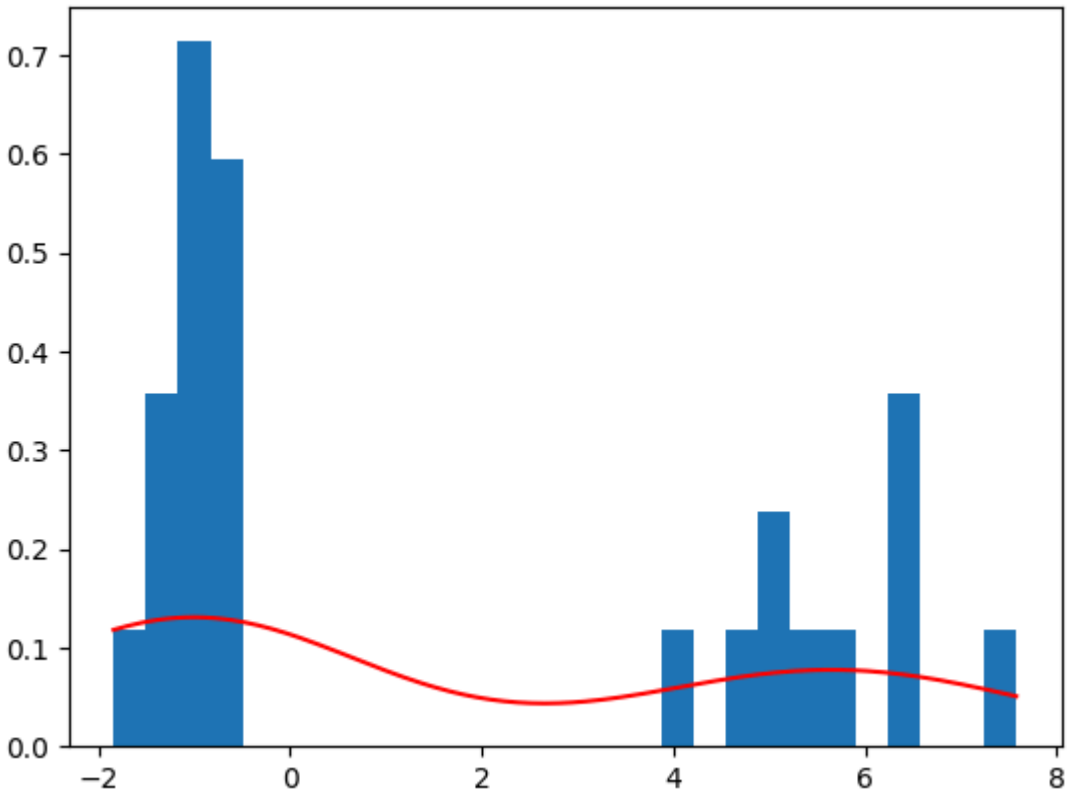
```
[<matplotlib.lines.Line2D at 0x7bc4ef382ad0>]
```



In [ ]:
```
from scipy.stats import kde
density = kde.gaussian_kde(y)
xgrid = np.linspace(x.min(), x.max(), 200)
plt.hist(y, bins=28, density=True)
plt.plot(xgrid, density(xgrid), 'r-')
```

```
/tmp/ipykernel_2666/3544470355.py:2: DeprecationWarning: Please import
`gaussian_kde` from the `scipy.stats` namespace; the `scipy.stats.kde`
namespace is deprecated and will be removed in SciPy 2.0.0.
  density = kde.gaussian_kde(y)
```

Out[112]:
```
[<matplotlib.lines.Line2D at 0x7bc4ef675e50>]
```

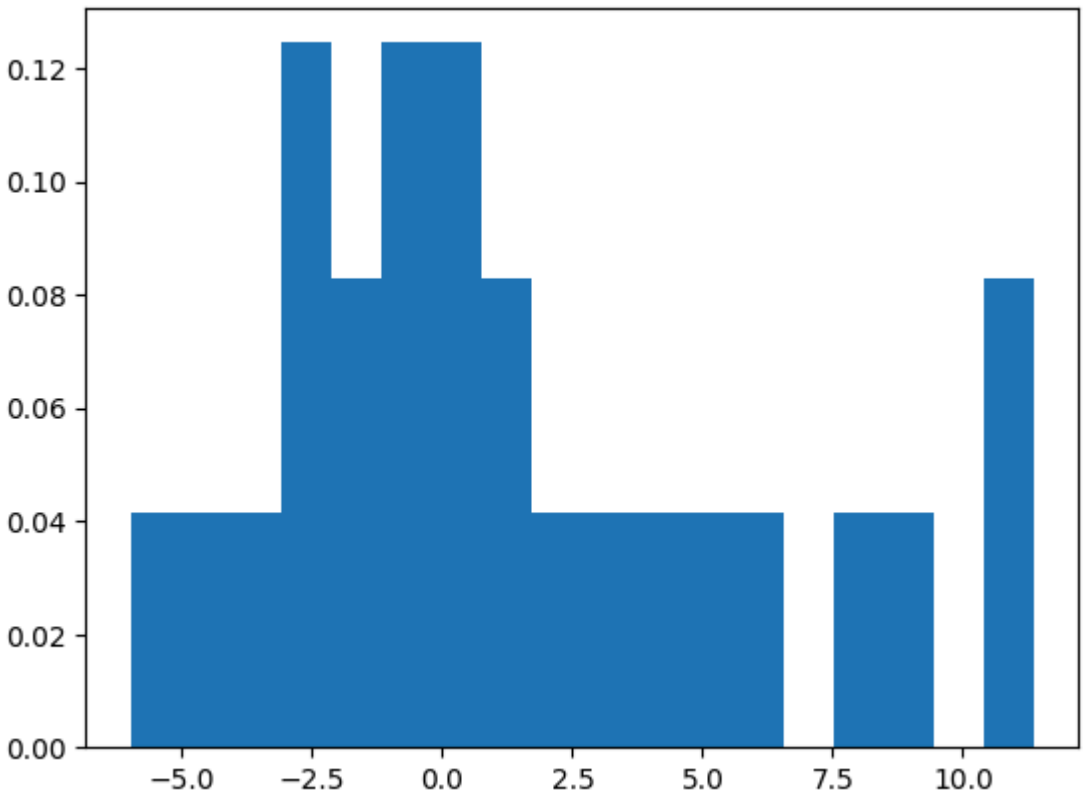


In [ ]:
```
# Create a bi-modal distribution with a mixture of Normals.
x1= np.random.normal(-1, 2, 15) # parameters: (loc=0.0, scale=1.0,
size=None)
x2 = np.random.normal(6, 3, 10)

# Append by row
x = np.r_[x1, x2] # r_ Translates slice objects to concatenation along the
first axis.
```

In [ ]:
```
plt.hist(x, bins=18, density=True)
```

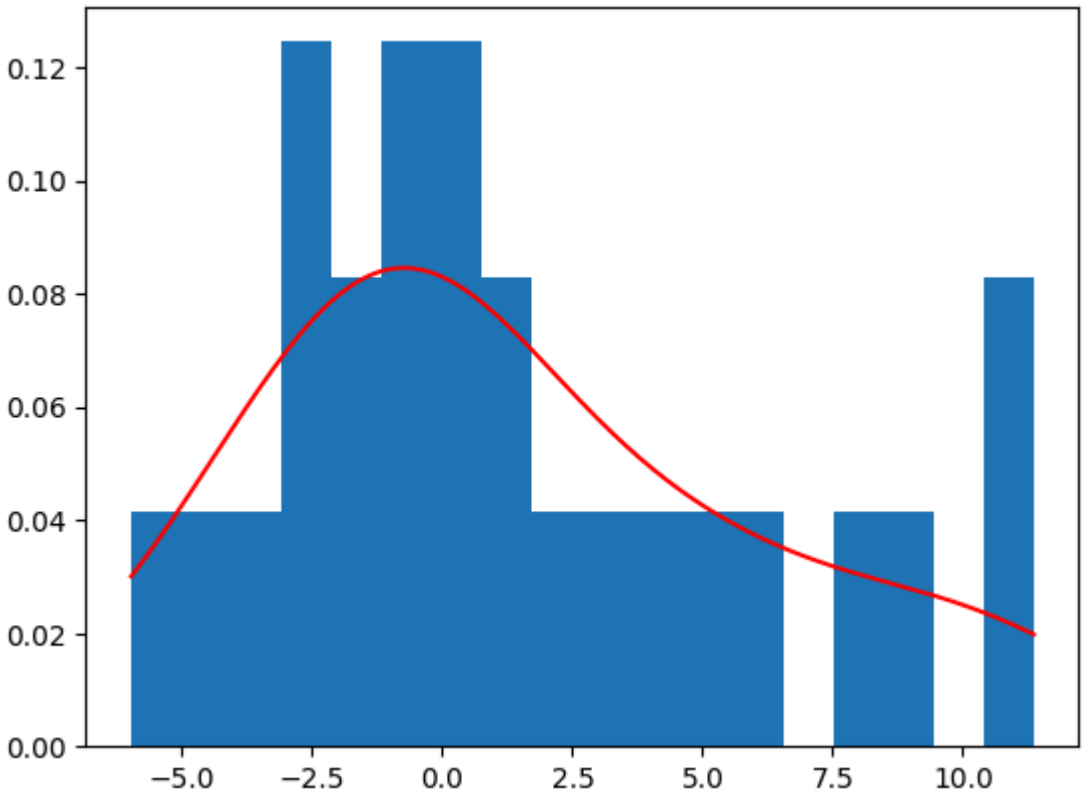
Out[116]:
```
(array([0.04150845, 0.04150845, 0.04150845, 0.12452535, 0.0830169 ,
        0.12452535, 0.12452535, 0.0830169 , 0.04150845, 0.04150845,
        0.04150845, 0.04150845, 0.04150845, 0.        , 0.04150845,
        0.04150845, 0.        , 0.0830169 ]),
 array([-5.97996427, -5.01630507, -4.05264587, -3.08898667, -2.12532747,
        -1.16166827, -0.19800907,  0.76565013,  1.72930933,  2.69296853,
         3.65662773,  4.62028693,  5.58394613,  6.54760533,  7.51126453,
         8.47492374,  9.43858294, 10.40224214, 11.36590134]),
 <BarContainer object of 18 artists>)
```



In [ ]:
```
density = kde.gaussian_kde(x)
xgrid= np.linspace(x.min(), x.max(), 200)
plt.hist(x, bins=18, density=True)
plt.plot(xgrid, density(xgrid), 'r-')
```

```
/tmp/ipykernel_2666/1753871923.py:1: DeprecationWarning: Please import
`gaussian_kde` from the `scipy.stats` namespace; the `scipy.stats.kde`
namespace is deprecated and will be removed in SciPy 2.0.0.
  density = kde.gaussian_kde(x)
```

Out[117]:
```
[<matplotlib.lines.Line2D at 0x7bc4ef418410>]
```



In [ ]:
```
# 13 - TREND ANALYSIS
# Gokul G (241801068)
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from statsmodels.graphics.tsaplots import plot_acf
from statsmodels.tsa.stattools import adfuller

df = pd.read_csv("/content/apple_weekly_2006_2018.csv")
df.head()
```

Out[120]: Columns 1–8 of 9

	Unnamed: 0	Open	High	Low	Close	Volume	Name
0	0	2.75	2.78	2.72	2.73	53641420	AAPL
1	1	2.70	2.74	2.69	2.71	60260557	AAPL
2	2	2.72	2.74	2.65	2.69	33804150	AAPL
3	3	2.73	2.76	2.67	2.67	95028169	AAPL
4	4	2.64	2.66	2.64	2.65	109092509	AAPL

Columns 9–9 of 9 · first column repeated

	Date
0	2006-01-06
1	2006-01-13
2	2006-01-20
3	2006-01-27
4	2006-02-03

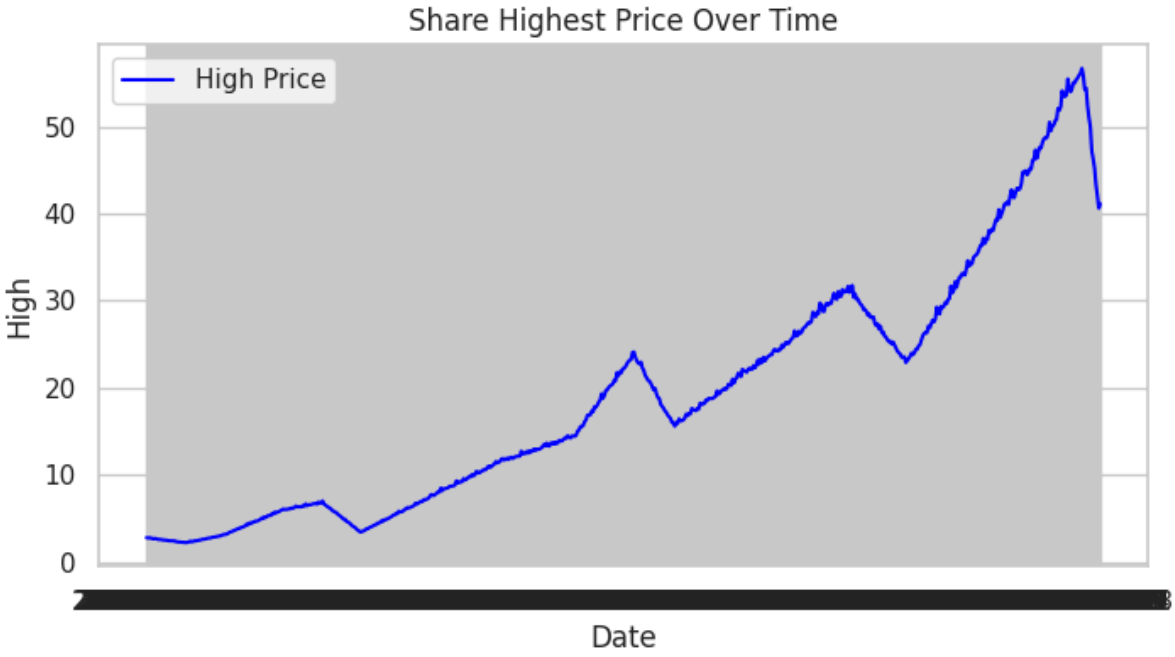
In [ ]:
```
df.drop(columns='Unnamed: 0', inplace =True)
df.head()
```

Out[121]:

	Open	High	Low	Close	Volume	Name	Date
0	2.75	2.78	2.72	2.73	53641420	AAPL	2006-01-06
1	2.70	2.74	2.69	2.71	60260557	AAPL	2006-01-13
2	2.72	2.74	2.65	2.69	33804150	AAPL	2006-01-20
3	2.73	2.76	2.67	2.67	95028169	AAPL	2006-01-27
4	2.64	2.66	2.64	2.65	109092509	AAPL	2006-02-03

In [ ]:

```
sns.set(style="whitegrid")
plt.figure(figsize=(8, 4))
sns.lineplot(data=df, x='Date', y='High', label='High Price', color='blue')
plt.xlabel('Date')
plt.ylabel('High')
plt.title('Share Highest Price Over Time')
plt.show()
```



In [ ]:

```
# Convert 'Date' to datetime and set it as index
df['Date'] = pd.to_datetime(df['Date'])
df_temp = df.set_index('Date')

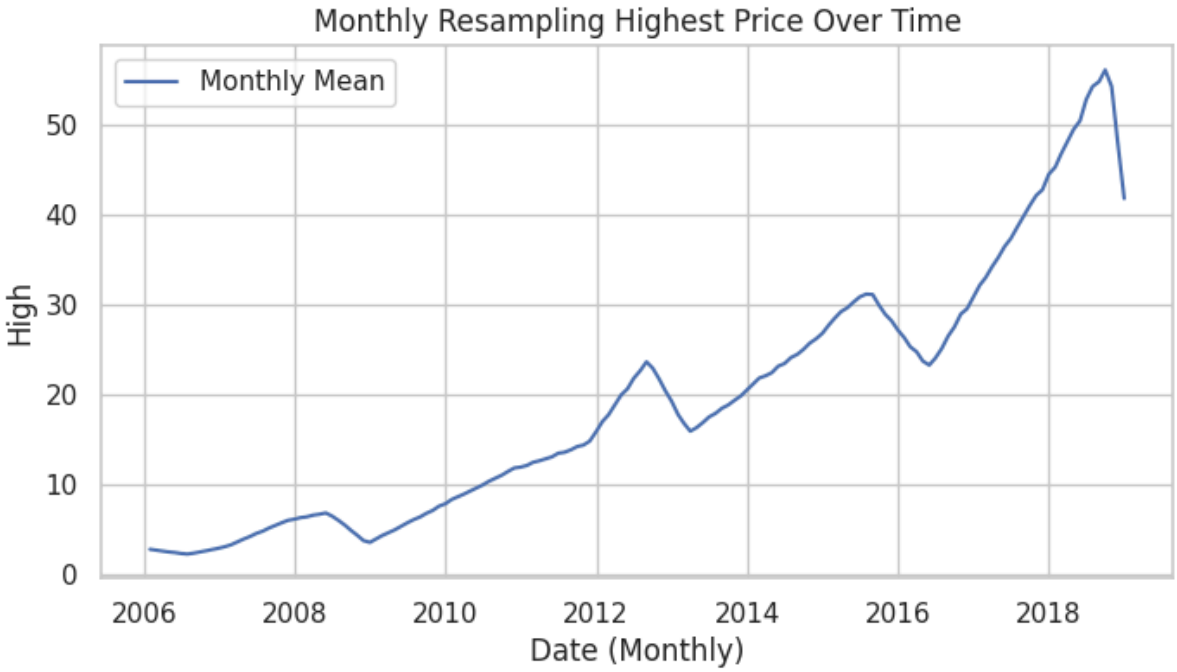
# Resample using 'ME' (Monthly End) as 'M' is deprecated
df_resampled = df_temp.resample('ME').mean(numeric_only=True)

sns.set(style="whitegrid")

plt.figure(figsize=(8, 4))
sns.lineplot(data=df_resampled, x=df_resampled.index, y='High',
label='Monthly Mean')

plt.xlabel('Date (Monthly)')
plt.ylabel('High')
plt.title('Monthly Resampling Highest Price Over Time')

plt.show()
```

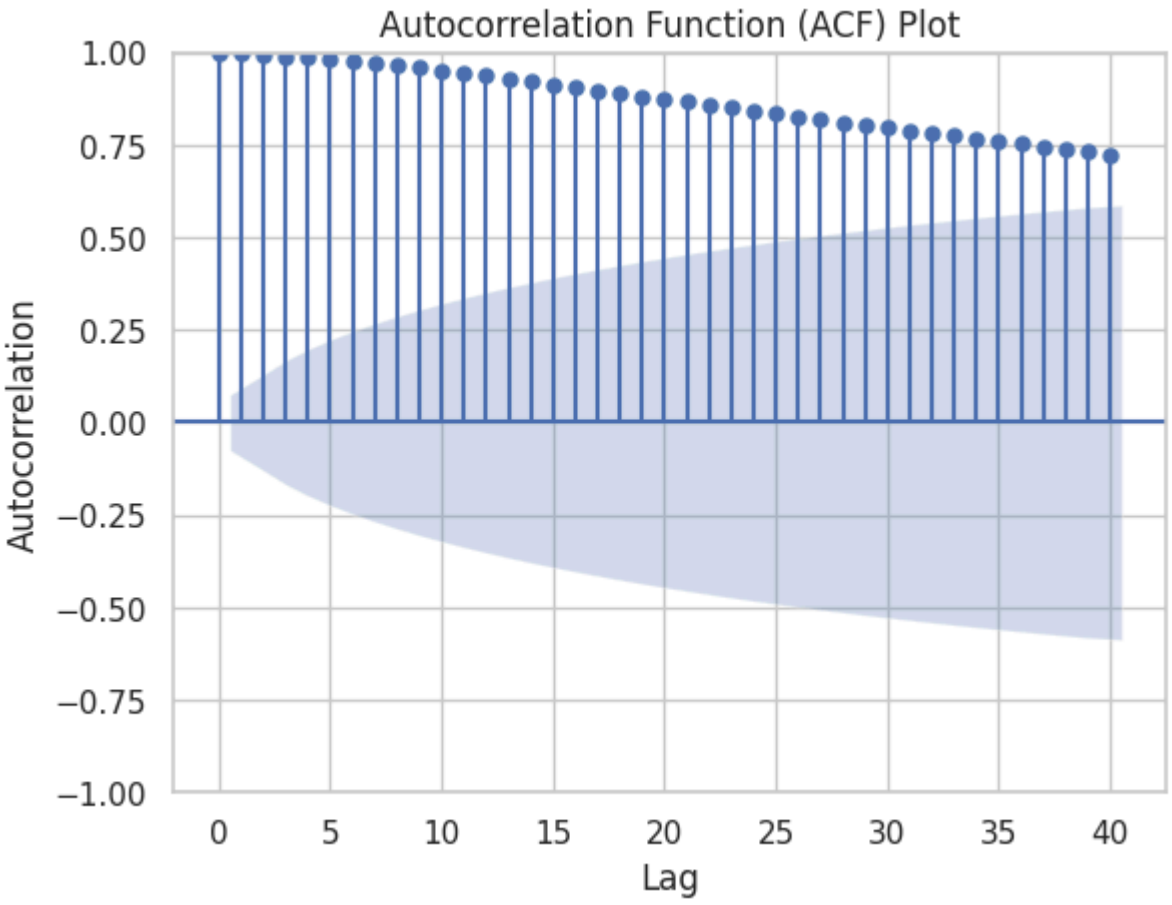


In [ ]:

```
if 'Date' not in df.columns:
  print("'Date' is already the index or not present in the DataFrame.")
else:
  df.set_index('Date', inplace=True)

plt.figure(figsize=(12, 6))
plot_acf(df['High'], lags=40)
plt.xlabel('Lag')
plt.ylabel('Autocorrelation')
plt.title('Autocorrelation Function (ACF) Plot')
plt.show()
```

```
<Figure size 1200x600 with 0 Axes>
```



In [ ]:
```
from statsmodels.tsa.stattools import adfuller

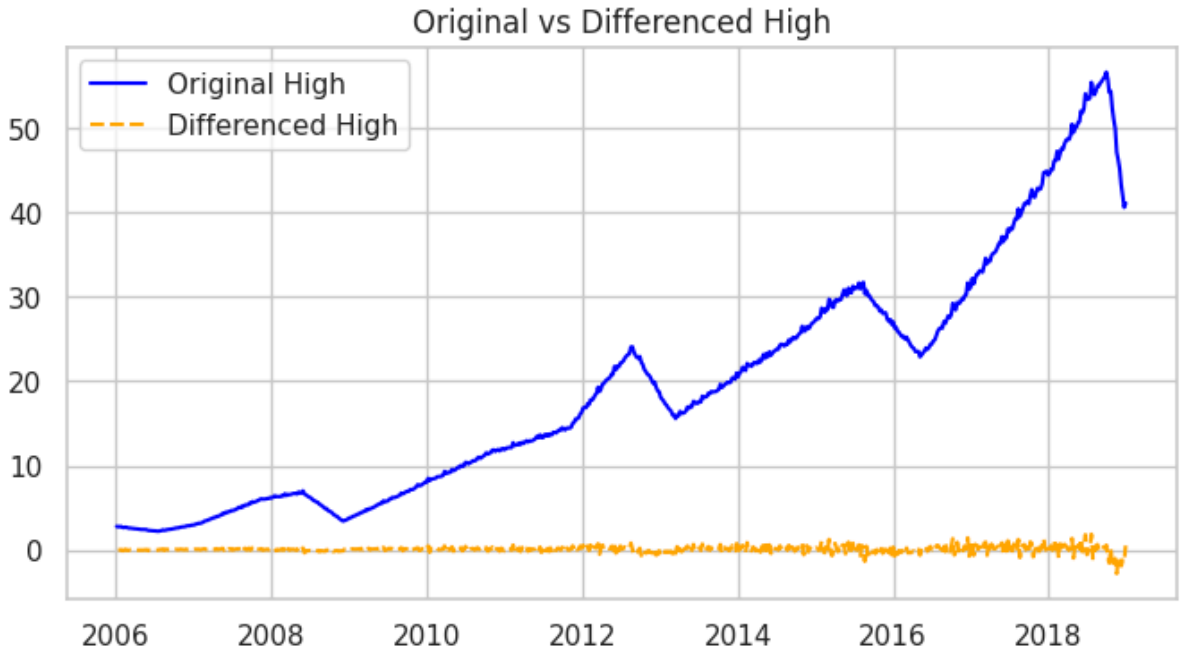
result = adfuller(df['High'])
print('ADF Statistic:', result [0])
print('p-value:', result [1])
print('Critical Values:', result [4])
```

```
ADF Statistic: -1.6104462120227558
p-value: 0.4777414008043818
Critical Values: {'1%': np.float64(-3.4402366871734196), '5%':
np.float64(-2.8659026013871864), '10%': np.float64(-2.5690932385868774)}
```

```
/tmp/ipykernel_2666/1211360421.py:3: FutureWarning: adfuller currently
returns a plain tuple whose length depends on the store and autolag
arguments. In release 0.16 or after July 2027, whichever is later, the
default behavior will switch to always returning an ADFullerResult. Set
result_object=True to switch now, or result_object=False to keep the current
behavior and silence this warning.
  result = adfuller(df['High'])
```

In [ ]:
```
df['high_diff'] = df['High'].diff()

plt.figure(figsize=(8, 4))
plt.plot(df['High'], label='Original High', color='blue')
plt.plot(df['high_diff'], label='Differenced High', linestyle='--',
color='orange')
plt.legend()
plt.title('Original vs Differenced High')
plt.show()
```

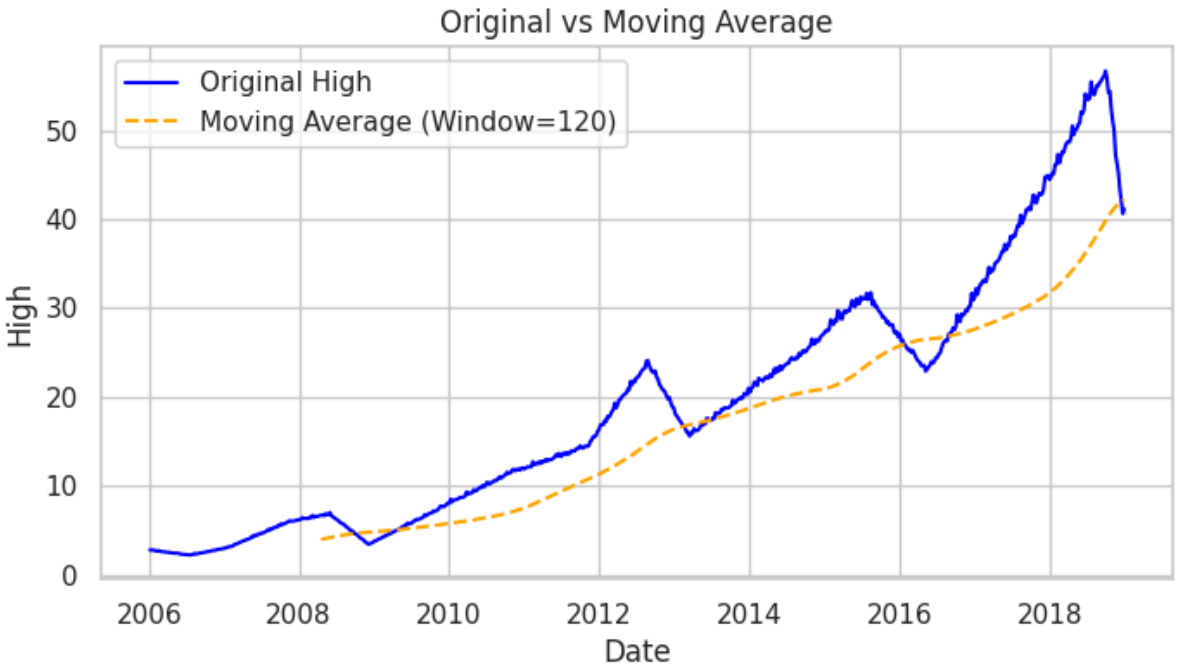


In [ ]:
```
window_size = 120
df['high_smoothed']=df['High'].rolling(window=window_size).mean()

plt.figure(figsize=(8, 4))

plt.plot(df['High'], label='Original High', color='blue')
plt.plot(df['high_smoothed'], label=f'Moving Average (Window=
{window_size})', linestyle='--', color='orange')

plt.xlabel('Date')
plt.ylabel('High')
plt.title('Original vs Moving Average')
plt.legend()
plt.show()
```



In [ ]:
```
df_combined = pd.concat([df['High'], df['high_diff']], axis=1)

print(df_combined.head())
```

```
            High  high_diff
Date
2006-01-06  2.78        NaN
2006-01-13  2.74      -0.04
2006-01-20  2.74       0.00
2006-01-27  2.76       0.02
2006-02-03  2.66      -0.10
```

In [ ]:
```
df.dropna(subset=['high_diff'], inplace=True)
df['high_diff'].head()
```

Out[136]:

Date	high_diff
2006-01-13	-0.04
2006-01-20	0.00
2006-01-27	0.02
2006-02-03	-0.10
2006-02-10	-0.02

dtype: float64

In [ ]:
```
from statsmodels.tsa.stattools import adfuller

result = adfuller(df['high_diff'])
print('ADF Statistic:', result [0])
print('p-value:', result [1] )
print('Critical Values:', result [4])
```

```
ADF Statistic: -2.973055509907004
p-value: 0.03748844810236733
Critical Values: {'1%': np.float64(-3.4402366871734196), '5%':
np.float64(-2.8659026013871864), '10%': np.float64(-2.5690932385868774)}
```

```
/tmp/ipykernel_2666/3240375043.py:3: FutureWarning: adfuller currently
returns a plain tuple whose length depends on the store and autolag
arguments. In release 0.16 or after July 2027, whichever is later, the
default behavior will switch to always returning an ADFullerResult. Set
result_object=True to switch now, or result_object=False to keep the current
behavior and silence this warning.
  result = adfuller(df['high_diff'])
```